Individuelle Praktische Arbeit

Weiterentwicklung Webapplikation für Incident Scanner

Autor: Scheibler Benjamin

Datum: 30. April 2025

Autor	Scheibler Benjamin
Datum	30. April 2025
Thema	Weiterentwicklung Webapplikation für Incident Scanner

geht an	Hauptexperte	Marco Lupi
	Nebenexperte	Livio Studach
	Verantwortliche Fachkraft	Laura Lenhardt
zur Kenntnis	Lernbegleiter	Cristina Ernst
	Fachliche Unterstützung	Susanne Sivakumaran

Version	1.0
Ersetzt Version	-
Ausgabedatum	[Veröffentlichungsdatum]
Gültig ab	[Veröffentlichungsdatum]
Gültig bis	-
Dokumentname	Dokument1

Inhaltsverzeichnis

Teil 1: Umfeld und Ablauf ... 9

1 Zielgruppe ... 10

1.1 Voraussetzungen an den Leser ... 10

2 Aufgabenstellung (gemäss PkOrg) ... 11

2.1 Ausgangslage ... 11
2.2 Detaillierte Aufgabenstellung ... 11
2.3 Deklaration der Vorkenntnisse ... 12
2.4 Neue Lerninhalte ... 12
2.5 Deklaration der Vorarbeiten ... 12

3 Deklaration der Firmenstandards ... 13

3.1 Unternehmensrichtlinien ... 13
3.2 Team Standards ... 13

4 Projektaufbauorganisation ... 14

4.1 Personen ... 14
4.1.1 Prüfungskandidat ... 14
4.1.2 Verantwortliche Fachkraft ... 14
4.1.3 Fachliche Unterstützung ... 14
4.1.4 Hauptexperte ... 15
4.1.5 Nebenexperte ... 15
4.2 Rollen und Aufgaben ... 16
4.2.1 Auftraggeber/Verantwortliche Fachkraft (VF) ... 16
4.2.2 Hauptexperte (HE) ... 16
4.2.3 Nebenexperte (NE) ... 16
4.2.4 Fachliche Unterstützung (FU) ... 16
4.2.5 IPA-Kandidat ... 16
4.3 Organigramm ... 17
4.4 Kommunikation ... 17

5 Organisation ... 18

5.1 Arbeitsplatz ... 18
5.1.1 Hardware ... 18
5.1.2 Software ... 18
5.1.3 Projektmethode ... 19
5.2 Datensicherung ... 19
5.2.1 Dokumentation ... 19
5.2.2 Quellcode ... 19

5.2.3 Datenbank ... 20
5.2.4 Backup-Recovery-Konzept ... 20
5.3 Projektphasen ... 21
5.3.1 Projektmanagement Überblick ... 21
5.3.2 Informieren ... 21
5.3.3 Planen ... 21
5.3.4 Entscheiden ... 21
5.3.5 Realisieren ... 21
5.3.6 Kontrollieren ... 21
5.3.7 Auswerten ... 21
5.4 Rapportierung ... 22

6 Ablauf ... 23

6.1 Ausführungsdaten und Termine ... 23
6.1.1 Arbeitstage ... 23
6.1.2 Termine ... 23

7 Zeitplan ... 24

8 Arbeitsjournal ... 25

8.1 Arbeitsjournal Tag 1 ... 25
8.2 Arbeitsjournal Tag 2 ... 26
8.3 Arbeitsjournal Tag 3 ... 27
8.4 Arbeitsjournal Tag 4 ... 28
8.5 Arbeitsjournal Tag 5 ... 29
8.6 Arbeitsjournal Tag 6 ... 30
8.7 Arbeitsjournal Tag 7 ... 31
8.8 Arbeitsjournal Tag 8 ... 32
8.9 Arbeitsjournal Tag 9 ... 33
8.10 Arbeitsjournal Tag 10 ... 34

9 Persönliches Fazit ... 35

9.1 Herausforderungen ... 35
9.2 Lernerfahrung ... 35
9.3 Entwicklungsperspektiven ... 35
9.4 Beurteilung des Erfolgs ... 35

Teil 2: Projekt-Dokumentation ... 36

10 Projektjournal ... 37

10.1 Erster Expertenbesuch ... 37
10.2 Besprechung Tag 1 ... 37
10.3 Besprechung Anforderungen ... 38
10.4 Besprechung Gestaltungsentwurf ... 38
10.5 Zweiter Expertenbesuch ... 39
10.6 Besprechung Stand IPA ... 39

11 Kurzfassung ... 40

11.1 Ausgangslage ... 40
11.2 Umsetzung ... 40
11.3 Ergebnis ... 40

12 Informieren ... 41

12.1 Anforderungen ... 41
12.1.1 Funktionale Anforderungen ... 41
12.1.2 Nicht-Funktionale Anforderungen ... 42
12.1.3 Legende Relevanz ... 42
12.2 Daten ... 43
12.2.1 Datenquellen ... 43
12.2.2 Datenbankstruktur ... 43
12.2.3 4V-Modell ... 44

13 Planen ... 45

13.1 Softwarearchitektur ... 45
13.2 Testkonzept ... 46
13.2.1 Testziele ... 46
13.2.2 Systemumgebung ... 46
13.2.3 Akteure ... 46
13.2.4 Daten ... 46
13.2.5 Testinfrastruktur ... 46
13.2.6 Abdeckung Anforderungen ... 47
13.2.7 Testfall T-01 Liste aller Cases API ... 48
13.2.8 Testfall T-02 Detailansicht Case API ... 49
13.2.9 Testfall T-03 Auflistung Cases Frontend ... 51
13.2.10 Testfall T-04 Detailansicht Case Frontend ... 52
13.2.11 Testfall T-05 Case mit Kommentar schliessen ... 53
13.2.12 Testfall T-06 Funktionsfähigkeit mit grosser Datenmenge ... 54
13.2.13 Testfall T-07 Korrektheit migrierter Daten ... 55
13.3 Migrationskonzept ... 56
13.3.1 Ziel ... 56
13.3.2 JSON-Report ... 56
13.3.3 Zuweisung JSON-Daten zur Datenbankstruktur ... 57
13.3.4 Mögliche Probleme ... 57
13.3.5 Optionen ... 57

13.4 Datenschutz ... 58
13.4.1 Klassifizierungsmodell ... 58
13.4.2 Klassifizierungsbaum ... 59
13.4.3 Klassifizierungstabelle ... 61
13.4.4 Klassifizierung Daten Tabellenebene ... 62
13.4.5 Klassifizierung Daten Spaltenebene ... 62
13.5 Rollenkonzept ... 63
13.5.1 Rollenbeschreibung ... 63
13.5.2 Zugriffskonzept ... 63
13.6 Gestaltungsentwurf ... 64
13.6.1 Übersichtsseite offene Cases ... 64
13.6.2 Übersichtsseite geschlossene Cases ... 65
13.6.3 Detailansicht ... 66
13.7 Realisierungskonzept ... 67
13.7.1 Use Cases und Abläufe ... 67
13.7.2 Komponenten ... 69
13.7.3 Schichten ... 69
13.7.4 Schnittstellen ... 70
13.7.5 Klassen ... 70
13.7.6 Datenmodell ... 71

14 Entscheiden ... 72

14.1 Entscheidungsmatrix ... 72
14.2 Entscheidungsmatrix Datenmigration ... 73
14.2.1 Varianten ... 73
14.2.2 Kriterien ... 74
14.2.3 Entscheidungsmatrix ... 74
14.3 API-Schnittstellen ... 75
14.3.1 Alle Cases abrufen ... 75
14.3.2 Case Details abrufen ... 75
14.3.3 Case schliessen ... 75
14.4 Entscheidungsmatrix Case schliessen Endpoint ... 76
14.4.1 Varianten ... 76
14.4.2 Kriterien ... 76
14.4.3 Entscheidungsmatrix ... 76
14.4.4 Endpoint Case schliessen ... 76

15 Realisieren ... 77

15.1 Auswahl ... 77
15.2 Setup ... 77
15.2.1 Datenbank ... 77
15.2.2 Entwicklungsumgebung ... 78
15.2.3 Python ... 78
15.2.4 Node.js ... 78
15.2.5 Git ... 78
15.2.6 Python Virtual Environment ... 79

15.2.7 Umgebungsvariablen ... 79
15.3 Datenmigration ... 80
15.3.1 Ablauf ... 80
15.3.2 Kontrolle in der Datenbank ... 80
15.3.3 Migrationsskript ... 81
15.4 Backend ... 83
15.4.1 Setup ... 83
15.4.2 Konfiguration ... 84
15.4.3 SwaggerUI Dokumentation ... 84
15.4.4 Datenbankverbindung ... 84
15.4.5 Alle Cases abrufen ... 85
15.4.6 Case Details abrufen ... 87
15.4.7 Case schliessen ... 89
15.4.8 Pagination ... 91
15.5 Frontend ... 93
15.5.1 Setup ... 93
15.5.2 Konfiguration ... 93
15.5.3 Offene Cases Übersicht ... 94
15.5.4 Geschlossene Cases Übersicht ... 94
15.5.5 Case Detailansicht ... 95
15.5.6 Case schliessen Modal ... 95
15.6 Softwareverwaltung ... 96

16 Kontrollieren ... 97

16.1 Datenmigration ... 97
16.1.1 Funktionstest ... 97
16.1.2 Performancetest ... 101
16.2 Testdurchführung API-Endpoints ... 102
16.2.1 Alle Cases abrufen ... 102
16.2.2 Detailansicht Cases ... 103
16.3 Testdurchführung ... 104

17 Auswerten ... 105

17.1 Fazit ... 105
17.2 Nacharbeiten ... 105

18 Kriterienkatalog ... 106

19 Verzeichnisse ... 107

19.1 Tabellenverzeichnis ... 107
19.2 Abbildungsverzeichnis ... 109
19.3 Quellenverzeichnis ... 110

20 Glossar ... 111

21 Anhang 112
21.1 Quellcode 112

Teil 1:
Umfeld und Ablauf

Swisscom (Schweiz) AG
Scheibler Benjamin
Förrlibuckstrasse 181
8005 Zürich
Mobile +41 79 841 95 00
E-Mail benjischeibler@gmail.com

1 Zielgruppe

1.1 Voraussetzungen an den Leser

Der Leser sollte ein solides Grundwissen im Bereich der Softwareentwicklung haben. Allen voran sollte er Kenntnis haben in Python, FastAPI, React, RESTful API. Die IPA ist so geschrieben, dass sie von einer Fachperson gelesen und verstanden werden kann.

2 Aufgabenstellung (gemäss PkOrg)

2.1 Ausgangslage

In der Security und Compliance ART werden verschieden Produkte entwickelt und betrieben, welche es unseren Kunden ermöglicht ihre Daten zu schützen. Eines dieser Produkte ist Sensitive Data Service, abgekürzt SDS, welches es ermöglicht verschiedene Datenablagen, Datenbanken, Fileablagen, etc. nach sensitiven Daten zu durchsuchen. Die dabei gefunden Stellen, an den solche Daten zu finden sind, werden zurzeit noch in einem Excel als Incidents zusammengefasst. Um diese Incidents unseren Kunden übersichtlicher verfügbar zu machen, ist die Umsetzung eines GUIs geplant, in dem das Anzeigen als auch das Management der Incidents ermöglicht wird.

2.2 Detaillierte Aufgabenstellung

Der Lernende programmiert ein GUI, welches es den Kunden erlaubt ihre Incidents, Fundstellen von sensitiven Daten, zu verwalten. Die Verwaltung der Incidents soll folgende Tätigkeiten beinhalten: - Offene (unbehandelte) Incidents anzeigen - Geschlossene Incidents anzeigen - Incident schliessen nach dem die sensitiven Stellen behoben wurden - Incident als falsch-positiv markieren, wodurch es geschlossen wird Um die IPA besser abgrenzen zu können, wird die Applikation zuerst unabhängig von der gesamten Produktlandschaft programmiert und nach Beendigung der IPA in diese integriert. Teil der IPA ist somit: Aufsetzen und befüllen der Datenbank - Postgres Datenbank aufsetzen mit vordefiniertem Datenbankmodell - Befüllen der Datenbank mit Testdaten --Es wird mit einem JSON File gearbeitet, das aus dem Report des SDS Scans direkt stammt. Bestandteil der IPA ist es, die Daten aus diesem JSON in die Datenbank zu migrieren. --Datenmigration kann zB. durch ein Python Skript geschehen, indem man die Daten des JSON in die DB migriert. zB. Mit Tools wie psycopg2. -Der Kandidat plant die Datensicherheit und den Datenschutz durch ein geeignetes Modell und klassifiziert die Daten. Backend mit den benötigten RestAPI Endpunkte erstellen -Identifizieren, welche Endpoints benötigt werden und Besprechung dieser mit den Auftraggebern (Fachvorgesetzte, technische Unterstützung) -Erstellen von Rest API Endpoints -Testen der Endpoints Frontend gemäss des erstellten Mock Ups umzusetzen - Erstellung der Mock Ups mit Figma und Besprechung dieser mit den Auftraggebern (Fachvorgesetzte, technische Unterstützung) - Übersichtsseite, die es ermöglicht entweder die offenen oder geschlossenen Incidents anzuzeigen - Detaillierte Ansichtsseite eines Incidents mit den Möglichkeiten dieses zu verwalten Funktionale Anforderungen: Die Incidents werden unterteilt in closed/open aufgelistet. Pagination wurde hinzugefügt. Die einzelnen Incidents lassen sich öffnen. Die Detailseite wird angezeigt: -Pfad -Dateiname -Kategorien --Pro Kategorie(Kategorie, zu welcher die Sensitive Daten dazugehörig sind) gibt es verschiedene Matches. Ein Match zeigt, wo die Sensitiven Daten gefunden wurden. --Pro Match sieht man Part, line, position -Button, um den Case mit einem Kommentar zu schliessen --Die Incidents lassen sich als "False-Positive" markieren und schliessen --Die Incidents lassen sich als "Behoben" markieren und schliessen Nicht-Funktionale Anforderungen: Die Applikation wird gemäss Firmenrichtlinien (https://peps.python.org/pep-0008/, https://sdx.swisscom.com/) umgesetzt. Der Lernende benutzt die Team-intern genutzten Commit Guidelines, welche in der IPA dokumentiert werden. Das Berechtigungsmanagement ist nicht Bestandteil der IPA. Die Applikation wird kein Login besitzen.

2.3 Deklaration der Vorkenntnisse

Legende: -- = Keine Erfahrung; - = ein wenig Erfahrung; + Erfahrung gesammelt; ++ = Lernender ist sattelfest in diesem Gebiet.

Python +,

FastAPI +,

Javascript ++,

React ++,

Postgres -

2.4 Neue Lerninhalte

Applikation mit FastAPI aufsetzen

Vertieftes Arbeiten mit einer Postgres Datenbank

2.5 Deklaration der Vorarbeiten

Firmeninterne IPA Vorlage, Datenmodell

3 Deklaration der Firmenstandards

3.1 Unternehmensrichtlinien

SDX (Swisscom Digital Experience) ist das Designsystem von Swisscom. Es besteht aus einer Komponentenbibliothek, UX-Prinzipien, Designrichtlinien, Dokumentationen und Regeln. SDX ist für alle Web-Apps oder Websites (sowohl extern als auch intern) obligatorisch, um sicherzustellen, dass alle Anwendungen unverwechselbar das einzigartige "Look and Feel" von Swisscom verkörpern. Die Swisscom hat auch Standards für Word Files und PowerPoint Präsentationen, welche ich für meine IPA anwenden werde.

3.2 Team Standards

Im Team gibt es Richtlinien für git. Die Branches werden nach Features und Bugfixes aufgeteilt. Der Branch Name startet demnach mit jeweils mit `feature/…` oder `bugfix/…`
Danach folgt die Jira Ticketnummer und darauf folgt eine Benennung des Branches in kebab-case.
Beispiel: `feature/SCSARES-129-ipa-branch`

Die Commit Messages müssen folgendermassen aussehen:
`Jira-Ticketnummer: Commit Nachricht im Imperativ`

4 Projektaufbauorganisation

4.1 Personen

4.1.1 Prüfungskandidat

Scheibler Benjamin	
Firma	Swisscom (Schweiz) AG
Strasse	Förrlibuckstrasse 181
PLZ, Ort	8005, Zürich
Tel. Nr.	079 841 95 00
E-Mail	benjischeibler@gmail.com

Tabelle 1 Informationen Lernender

4.1.2 Verantwortliche Fachkraft

Lenhardt Laura	
Firma	Swisscom (Schweiz) AG
Strasse	Förrlibuckstrasse 181
PLZ, Ort	8005, Zürich
Tel. Nr.	0796266415
E-Mail	Laura.Lenhardt@swisscom.com

Tabelle 2 Information Verantwortliche Fachkraft

4.1.3 Fachliche Unterstützung

Susanne Sivakumaran	
Firma	Swisscom (Schweiz) AG
Strasse	Förrlibuckstrasse 181
PLZ, Ort	8005, Zürich
Tel. Nr.	+41 79 681 95 80
E-Mail	Susanne.Sivakumaran@swisscom.com

Tabelle 3 Information Fachliche Unterstützung

4.1.4 Hauptexperte

Lupi Marco	
Tel. Nr.	0796503341
E-Mail	lupi.marco@gmail.com

Tabelle 4 Information Hauptexperte

4.1.5 Nebenexperte

Studach Livio	
Tel. Nr.	+41 79 532 15 49
E-Mail	livio.studach@gmail.com

Tabelle 5 Information Nebenexperte

4.2 Rollen und Aufgaben

4.2.1 Auftraggeber/Verantwortliche Fachkraft (VF)

Die verantwortliche Fachkraft ist für die Beschreibung des Auftrags zuständig. Ausserdem übernimmt sie alle administrativen Aufgaben im Zusammenhang mit der IPA. Die VF wird nach der IPA zusammen mit den Experten die Bewertung besprechen. Die VF ist zudem auch Auftraggeber und über das Projekt hindurch der einzige Stakeholder.

4.2.2 Hauptexperte (HE)

Der Hauptexperte stellt den korrekten Ablauf der IPA sicher. Er bewertet am Ende das Ergebnis und führt das Fachgespräch durch. Er steht dem IPA-Kandidat und der VF jederzeit für Fragen bezüglich der IPA zur Verfügung. Während der IPA fragt er Besuchstage an, wo er die VF und den IPA-Kandidat über das weitere Vorgehen informiert.

4.2.3 Nebenexperte (NE)

Der Nebenexperte unterstützt den Hauptexperten bei dessen Aufgaben während der IPA. Er ist ebenfalls an der Bewertung sowie am Fachgespräch beteiligt und bringt dabei eine zusätzliche Perspektive in die Beurteilung ein.

4.2.4 Fachliche Unterstützung (FU)

Die fachliche Unterstützung steht dem IPA-Kandidaten bei allfälligen technischen Fragen und Problemen während der IPA zur Seite.

4.2.5 IPA-Kandidat

Der IPA-Kandidat arbeitet an dem Produkt, dokumentiert seine Schritte und Erkenntnisse in dieser Dokumentation und ist am Ende der Bewertete. Nach Abgabe des Produkts wird er das erarbeitete Produkt demonstrieren und sich im anschliessenden Fachgespräch den Fragen des HE stellen.

4.3 Organigramm

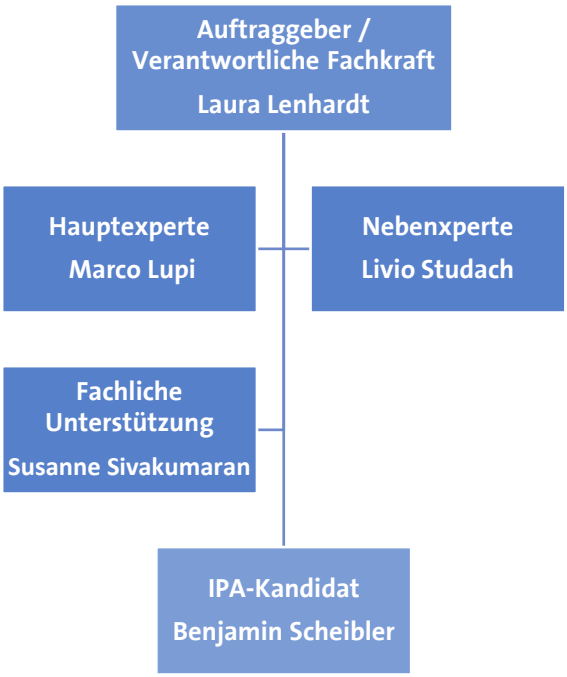


Abbildung 1 Organigramm

4.4 Kommunikation

Die Kommunikation innerhalb des Teams, sprich IPA-Kandidat, VF und FU, läuft über das Kommunikationstool Microsoft Teams ab oder gleich vor Ort in Person. Die geplanten Gespräche zwischen IPA-Kandidat und VF sorgen für eine proaktive, konstruktive Kommunikation, um allfällige Unklarheiten schnellstmöglich zu bereinigen und mögliche Risiken zu identifizieren. Die Inhalte der Gespräche sind konstruktiv und verlaufen effizient ab, ohne abzuschweifen.

Die Kommunikation mit dem Experten läuft über E-Mail oder Telefon ab.

5 Organisation

5.1 Arbeitsplatz

Abbildung 2 Arbeitsplatz

Der Arbeitsplatz befindet sich in einem eigenen Raum, sodass ich nicht gestört werde und mich voll meiner Arbeit, während den zehn Tagen widmen kann. Die Kriterien sind an der Wand angebracht, damit diese fortlaufend aktuell gehalten werden.

5.1.1 Hardware

- HP Monitor
- HP Elitebook 860 G11
- Samsung Galaxy s21+ (Geschäftshandy)
- Logitech Tastatur
- Sygonix Maus

5.1.2 Software

- Windows 11
- Visual Studio Code 1.99.2
- PostgreSQL Version 19
- DBeaver Community Version
- Mozilla Firefox Webbrowser
- Postman 11.40.6
- Git 2.47.1
- Gitlab
- Figma

5.1.3 Projektmethode

Ich habe mich dafür entscheiden die IPA mit der Projektmethode IPERKA zu absolvieren.

Die Wahl der IPERKA-Methode ist sinnvoll, da sie einen klar strukturierten und schrittweisen Arbeitsablauf bietet, der alle relevanten Aspekte der Entwicklung abdeckt. Gerade bei einem Kleinprojekt wie diesem ist es entscheidend, den Überblick zu behalten und zielgerichtet vorzugehen. Die Informieren-Phase hilft mir, ein umfassendes Bild vom Projekt zu machen und die Anforderungen sowie den Projektumfang besser zu verstehen. Dadurch kann ich die weiteren Schritte gezielt planen und strukturiert umsetzen.

5.2 Datensicherung

5.2.1 Dokumentation

Die Dokumentation befindet sich in einem OneDrive Ordner, welcher von der Swisscom zur Verfügung gestellt wird. Ich habe den Toggle "Automatisches Speichern" aktiviert, wodurch meine IPA fortlaufend bei mir lokal und in der Cloud gespeichert wird. Da ich immer Internetverbindung habe, synchronisiert OneDrive die Dateien automatisch. Falls ich aus irgendwelchen Gründen keine Internetverbindung mehr haben sollte, ist die Datei trotzdem noch lokal gespeichert und wird mit OneDrive synchronisiert, sobald ich wieder mit dem Internet verbunden bin. Ich erstelle jeden Tag eine neue Kopie des Dokumentes, falls die Hauptdatei beschädigt werden sollte. Ausserdem lösche ich die Kopie des Vortages, um eine gewisse Ordnung zu halten. Anhand des Status sehe ich, ob die Datei synchronisiert wurde.

Name	Status	Änderungsdatum	Typ
IPA - Kopie Tag 3.docx		17.04.2025 16:08	Microsoft Word-Dokum...
IPA.docx		17.04.2025 16:08	Microsoft Word-Dokum...

Abbildung 3 Datensicherung Dokumentation

Den gleichen Ablauf habe ich ebenfalls mit dem Zeitplan.

IPA_Zeitplan.xlsx		30.04.2025 11:34	Microsoft Excel-Arbeits...
IPA_Zeitplan - Kopie.xlsx		30.04.2025 11:34	Microsoft Excel-Arbeits...

Abbildung 4 Datensicherung Zeitplan

5.2.2 Quellcode

Der Quellcode wird in einem Swisscom-internen Gitlab Repository gespeichert. Dieses klone ich lokal, sodass es auch auf meinem Gerät gespeichert ist. Durch regelmässige Commits ist der Quellcode in Gitlab stets aktuell.

5.2.3 Datenbank

Bei Änderungen in der Datenbank, wird regelmässig ein Dump erstellt, welcher dann ebenfalls im OneDrive Ordner abgelegt wird und so doppelt gesichert wird. Falls die Datenbank aus irgendwelchen Gründen ausfällt, kann ich die Daten durch den gesicherten Dump innert Minuten wiederherstellen. Ausserdem könnte man die Datensicherung durch das Backup-Recovery-Konzept, welches im nächsten Kapitel beschrieben wird, ergänzen.

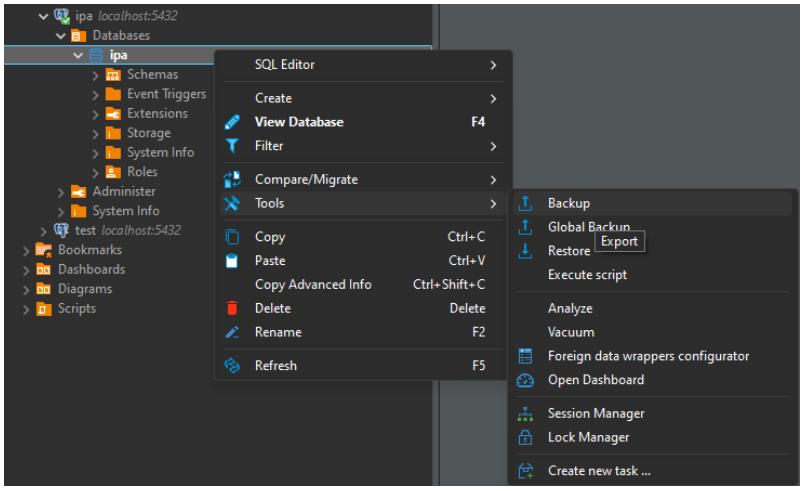


Abbildung 5 Datenbank Backup

5.2.4 Backup-Recovery-Konzept

Regelmässige Vollbackups

Tägliche vollständige Backups der Datenbank erstellen. Hierfür kann das Tool "pg_dump" verwendet werden oder man kann einen Dump in DBeaver erstellen.

Automatisierung von Backups

Das Erstellen der Postgres-Dumps kann man automatisieren, indem man ein Batch-Skript mit folgendem Befehl schreibt:

```
pg_dump -U postgres -F c -f C:\IPA\Backup_folder\backup_%date%.dump db_ipa
```

Dieses Skript kann man dann durch den Task Scheduler von Windows in dem festgelegten Intervall ausführen lassen.

Sicherer Speicherort

Backup-Dateien sollten nicht ausschliesslich lokal gespeichert werden, sondern auch auf einem externen Speicher, wie der beschriebene OneDrive Ordner.

Recovery

Um ein Backup wiederherzustellen, kann man das Tool "pg_restore" verwenden. Bei einem Backup im Custom-Format (`.dump`) lautet der Befehl zum Beispiel:

```
pg_restore -U postgres -d db_ipa -c
C:\IPA\Backup_folder\backup_%date%.dump
```

Dabei sollte sichergestellt sein, dass die Ziel-Datenbank existiert oder mit dem Restore neu erstellt wird. Alternativ kann man in DBeaver einen Dump auch grafisch erstellen (rechte Maustaste auf Datenbank → Tools → Restore).

5.3 Projektphasen

In diesem Kapitel wird die gewählte Projektmethode IPERKA und deren Phasen genauer beschrieben und dargestellt.

(ICT Berufsbildung Bern, 2025)

5.3.1 Projektmanagement Überblick

Abbildung 6 Projektmanagement

5.3.2 Informieren

In dieser Phase geht es darum, den Auftrag und das angestrebte Ziel genau zu verstehen. Systematisch werden alle relevanten Informationen gesammelt, sortiert und bewertet. Es werden zentrale Fragen gestellt, um den Rahmen und die Anforderungen der Aufgabe zu verstehen.

5.3.3 Planen

Auf Basis der gesammelten Informationen werden in dieser Phase verschiedene Lösungsvarianten erarbeitet und das weitere Vorgehen strukturiert. Arbeitsschritte werden definiert, zeitlich eingeordnet und die benötigten Ressourcen identifiziert. Ausserdem werden Konzepte erarbeitet.

5.3.4 Entscheiden

Nachdem verschiedene Lösungsvarianten entwickelt wurden, erfolgt die Auswahl der besten Option. Entscheidungen werden möglichst zügig und unter Einbeziehung aller Beteiligten getroffen, um Zeitverluste zu vermeiden. Wichtige Kriterien und Erfahrungen fliessen in den Entscheidungsprozess ein, sodass die gewählte Lösung realisierbar und zielgerichtet ist.

5.3.5 Realisieren

In der Realisierungsphase wird der erstellte Arbeitsplan praktisch ausgeführt. Dabei ist es wichtig, sich an die Planung zu halten und die Dokumentation zu aktualisieren. Sollte es zu Abweichungen oder Problemen kommen, müssen diese zeitnah identifiziert, korrigiert und dokumentiert werden, um den Erfolg des Projekts sicherzustellen.

5.3.6 Kontrollieren

Nach der Umsetzung wird überprüft, ob das Ergebnis dem festgelegten Plan entspricht. Dies erfolgt durch den Vergleich mit den ursprünglichen Anforderungen des Testkonzepts. Fehler oder Mängel werden dokumentiert und notwendige Korrekturmassnahmen eingeleitet, bevor das Projekt vollständig abgeschlossen wird.

5.3.7 Auswerten

In der abschliessenden Reflexion wird der gesamte Prozess und Dokumentation analysiert. Dabei wird bewertet, was gut funktionierte und wo es Verbesserungspotenzial gibt. Die Auswertungsphase geschieht ausserdem zusätzlich durch den HE und dem NE, welche die IPA-Dokumentation bewerten.

5.4 Rapportierung

Während der Durchführung der IPA wird täglich ein Arbeitsjournal geführt, um die Übersicht des Projektes zu behalten und den Fortschritt zu dokumentieren. Im Arbeitsjournal werden stichwortartig oder in kurzen Sätzen folgende Themen erfasst:

- Ausgeführte Arbeiten mit Soll / Ist vergleich nach Zeitplan
- Erreichte Ziele
- Aufgetretene Probleme
- Tests
- Hilfestellung
- Vergleich Zeitplan
- Positives
- Negatives
- Reflexion

6 Ablauf

6.1 Ausführungsdaten und Termine

6.1.1 Arbeitstage

Tag	Datum
1	Dienstag, 15.04.2025
2	Mittwoch, 16.04.2025
3	Donnerstag, 17.04.2025
4	Dienstag, 22.04.2025
5	Mittwoch, 23.04.2025
6	Donnerstag, 24.04.2025
7	Freitag, 25.04.2025
8	Montag, 28.04.2025
9	Dienstag, 29.04.2025
10	Mittwoch, 30.04.2025

Tabelle 6 Arbeitstage

6.1.2 Termine

Titel	Datum	Zeit	Ort	Teilnehmer
IPA-Beginn	Dienstag, 15.04.2025	08:30	Zürich	Benjamin Scheibler
Erster Besuchstag	Dienstag, 15.04.2025	15:30	Remote	Marco Lupi, Livio Studach, Laura Lenhardt, Benjamin Scheibler
Zweiter Expertenbesuch	Montag, 28.04.2025	16:00	Zürich	Marco Lupi, Livio Studach, Laura Lenhardt, Benjamin Scheibler
IPA-Abgabe	Mittwoch, 30.04.2025	18:00	Zürich	Benjamin Scheibler
IPA-Präsentation & Fachgespräch	Freitag, 16.05.2025	09:00	Zürich	Marco Lupi, Livio Studach, Laura Lenhardt, Benjamin Scheibler

Tabelle 7 Termine

7 Zeitplan

Zeitplan Dauer: 87h IPA Kandidat: Benjamin Scheibler Projekt: Weiterentwicklung Webapplikation für Incident Scanner	Aufwand SOLL in h	Aufwand IST in h	15.04.2025				16.04.2025				17.04.2025				22.04.2025				23.04.2025				24.04.2025				25.04.2025				28.04.2025				29.04.2025				30.04.2025			
Aktivitäten			2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h	2h
1. Teil: Umfeld und Ablauf	**7.5**	**5.5**																																								
Administratives Dokumentation	0.5	1.0	0.5	1.0																																						
Firmenstandards	1.0	0.5	1.0	0.5																																						
Projektaufbauorganisiation	2.0	1.0	0.5	1.5	1.0																																					
Organisation	4.0	3.0		0.5	1.5	2.0																																				
					2.0	1.0																																				
2. Teil: Projekt-Dokumentation	**2.0**	**3.0**																																								
Kurzfassung	2.0	3.0					2.0																																1.0			
						2.0																																				
1.Informieren	**3.5**	**2.0**																																								
Anforderungen definieren und besprechen	2.0	1.0						2.0																																		
Daten identifizieren und bewerten	1.5	1.0					1.0		1.5																																	
2. Planen	**21.5**	**16.5**																																								
Zeitplan erstellen	2.0	1.5						0.5	1.5																																	
Softwarearchitektur	3.0	2.0						1.5			2.0	1.0																														
Testkonzept	4.0	3.0						0.5	1.5			1.0	2.0	1.0																												
Migrationskonzept	1.5	1.5						0.5	2.5					0.5	1.0																											
Datenschutz	3.0	2.5									1.5																															
Gestaltungsentwurf	3.0	2.5									0.5	2.0			1.0	2.0																										
Realisierungskonzept	5.0	3.5										2.0	1.5					1.0	2.0	2.0																						
3. Entscheiden	**3.0**	**2.0**																																								
Entscheidungsmatrizen	3.0	2.0													2.0						2.0	1.0																				
4. Realisieren	**32.5**	**36.5**																																								
Datenbank aufsetzen	2.0	1.0														1.0					1.0	1.0																				
Datenmigration	2.0	2.0														1.0	1.0						1.0	1.0																		
Backend und Frontend aufsetzen	2.5	2.5																1.0	1.5						1.0	1.5																
Backend endpoints implementieren	12.0	10.0																			1.5	2.0	2.0	2.0	2.0	1.0			1.5													
Frontend umsetzen	8.0	12.5																	1.5	2.0	2.0		1.0	2.0	2.0		2.0	2.0	2.0	1.0			2.0	2.0	2.0	2.0	0.5	2.0				
Testing	4.0	6.0																													2.0	2.0					2.0	2.0				
Bugfixing	2.0	2.5																	0.5				0.5				1.0				0.5								2.0			
5. Kontrollieren	**3.0**	**5.0**																																								
Testinfrastruktur	1.0	2.0																													0.5	1.5							1.0			
Testdurchführung	2.0	3.0																													2.5						0.5			2.0		
6. Auswerten	**2.5**	**2.0**																																								
Nacharbeiten definieren	0.5	0.5																																					0.5			
Persönliches Fazit	2.0	1.5																																						2.0		
																																								1.5		
7. Verschiedenes & Wiederholendes	**11.0**	**14.5**																																								
Arbeitsjournal	5.0	5.0		0.5	0.5			0.5	0.5			0.5	0.5		0.5	0.5			0.5	0.5			0.5	0.5			0.5	0.5			0.5	0.5			0.5	0.5				0.5	0.5	
Lücken in der Dokumentation füllen	2.0	5.5						0.5				0.5	0.5					0.5					0.5				0.5								0.5	1.5			0.5	0.5	2.0	
Expertenbesuche	2.0	2.0			1.0	1.0																									1.0	1.0										
Reserve	2.0	2.0																																							2.0	2.0
Total Aufwand SOLL	**86.5**		9.0				8.5				8.5				8.5				8.5				9.0				8.5				9.0				8.5				8.5			
Total Aufwand IST		**87.0**	9.0				9.0				9.0				8.5				8.5				9.0				8.0				9.0				9.0				8.0			
Differenz	**0.5**																																									

Legende	
Stunden	Aufwand SOLL
Stunden	Aufwand IST
Stunden	Expertenbesuche

Swisscom (Schweiz) AG
Scheibler Benjamin
Förrlibuckstrasse 181
8005 Zürich
Mobile +41 79 841 95 00
E-Mail benjischeibler@gmail.com

8 Arbeitsjournal

8.1 Arbeitsjournal Tag 1

Datum	Dienstag, 15.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 8.5	Soll	Ist
	• Administratives in Dokumentation	0.5	1
	• Firmenstandards	1	0.5
	• Projektaufbauorganisation	2	1
	• Kapitel Organisation	4	3
	• Kurzfassung	2	2
	• Erster Expertenbesuch	1	1
	Total in Stunden	10.5	8.5
Erreichte Ziele	• Ich bin mit Teil 1 der Dokumentation fertig und konnte bereits mit der Informieren Phase beginnen		
Aufgetretene Probleme	• Keine		
Tests	• Keine		
Hilfestellungen	• Ich habe die geschriebenen Texte von ChatGPT überprüfen und überarbeiten lassen. (OpenAI, 2025)		
Vergleich Zeitplan	• Die Kurzfassung war für Tag 2 geplant. Da ich mit den vorherigen Kapiteln aber bereits früher als geplant fertig war, konnte ich bereits die Kurzfassung schreiben. Deshalb ist auch oben der soll Bereich bei 10.5h • Der Expertenbesuch habe ich vergessen einzurechnen.		
Positives	• Ich bin weiter vorangekommen als geplant		
Negatives	• Zeitplanung lief nicht wie erwartet, da die Planung zu unpräzis war. • Ich habe die Kurzfassung jetzt schon geschrieben, was nach Absprache mit der VF nicht viel Sinn ergibt. Ich werde die Kurzfassung am letzten Tag nochmals überarbeiten, da ich diese dann genauer schreiben kann.		
Reflexion	• Ich habe die Planung zu wenig genau angeschaut und den Expertenbesuch nicht miteingerechnet. Ausserdem war das Excel an gewissen Stellen nicht korrekt. Ich lerne daraus, dass ich mir für die Planung nächstes Mal mehr Zeit einplane, diese Schritt für Schritt durchgehe und nach Korrektheit prüfe. Aus diesem Grund werde ich mir morgen Zeit nehmen, um den Zeitplan nochmals zu überarbeiten.		

Tabelle 8 Arbeitsjournal Tag 1

8.2 Arbeitsjournal Tag 2

Datum	Mittwoch, 16.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 9	Soll	Ist
	• Anforderungen	2	1
	• Daten identifizieren und bewerten	1.5	1
	• Zeitplan	2	1.5
	• Softwarearchitektur	3	2
	• Testkonzept	4	3
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	13	9
Erreichte Ziele	• Ich konnte den Zeitplan revidieren, Fehler beheben und die ganzen zehn Tage planen • Anforderungen sind definiert • Informationsphase abgeschlossen		
Aufgetretene Probleme	• Keine		
Tests	• Keine		
Hilfestellungen	• Ich nutzte das online Tool PlantUML, um das Softwarearchitekturdiagramm zu erstellen (PlantUML Web Server, n.d.) • Ausserdem benutzte ich ChatGPT, um meine Tests zu überarbeiten, verständlicher zu schreiben und auf Rechtschreibung zu prüfen (OpenAI, 2025) • Mit meiner VF zusammen sind wir ich die Anforderungen durchgegangen, haben diese validiert und überarbeitet.		
Vergleich Zeitplan	• Ich bin sehr gut unterwegs im Zeitplan, da ich heute richtig einen "Flow" beim Schreiben der Dokumentation hatte. Ich werde also früher als geplant zum "Realisieren" Teil vordringen, wo vermutlich mehr Probleme auftreten werden, die mehr Zeit in Anspruch nehmen als geplant.		
Positives	• Sehr gut im Zeitplan, einiges schneller abgeschlossen als geplant.		
Negatives	• Nichts		
Reflexion	• Ich habe gemerkt, wie wichtig ein gut strukturierter Zeitplan ist und bin froh diesen jetzt zu haben. Ich bin sehr zufrieden mit meinen Leistungen heute und mit dem, was ich erreicht habe. Wenn ich so weiterarbeite, kommt das sehr gut.		

Tabelle 9 Arbeitsjournal Tag 2

8.3 Arbeitsjournal Tag 3

Datum	Donnerstag, 17.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 9	Soll	Ist
	• Migrationskonzept	1.5	1.5
	• Abschnitt Datenschutz	3.0	2.5
	• Gestaltungsentwurf	0.5	0.5
	• Realisierungskonzept	5.0	3.5
	• Lücken Dokumentation	0.5	0.5
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	11	9
Erreichte Ziele	• Das Migrationskonzept und das Realisierungskonzept sind beide fertig. • Zusätzlich ist der Abschnitt Datenschutz auch schon erledigt, was heisst, dass in der Phase "Planen" nur noch das Gestaltungskonzept fehlt, an dem ich bereits begonnen habe zu arbeiten.		
Aufgetretene Probleme	• Keine		
Tests	• Keine		
Hilfestellungen	• Ich nutzte ChatGPT, um meine Texte zu überarbeiten, verständlicher zu schreiben und auf Rechtschreibung zu prüfen. • Ausserdem habe ich jeweils die einzelnen Komponenten im Realisierungskonzept geschrieben und mir versucht ein UML-Diagramm erzeugen zu lassen.		
Vergleich Zeitplan	• Ich bin sehr gut im Zeitplan. Ich bin mit den Phase Planen fast fertig, obwohl geplant ist, dass diese nach Tag 4 fertig sein soll.		
Positives	• Gut im Zeitplan • Phase "Planen" fast abgeschlossen • Keine grösseren Probleme • Alle Konzepte geplant		
Negatives	• Nichts		
Reflexion	• Ich bin auch heute wieder äusserst zufrieden mit meiner Leistung. Ich hoffe ich kann diesen Flow über das Wochenende mitnehmen und am Dienstag so weiterarbeiten. • Die Konzepte konnte ich nachvollziehbar und effizient ausarbeiten, was mir in der Realisierungsphase sicherlich helfen wird. • Rückblickend war der Wiedereinstieg in die Arbeit mit PlantUML harziger als gedacht. Nach ein wenig Übung und Hilfe von ChatGPT bin ich aber wieder reingekommen		

Tabelle 10 Arbeitsjournal Tag 3

8.4 Arbeitsjournal Tag 4

Datum	Dienstag, 22.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 8.5h	Soll	Ist
	• Gestaltungsentwurf	2.5	2
	• Phase "Entscheidung"	3	2
	• Datenbank aufsetzen	2	1
	• Datenmigration	2	2
	• Backend und Frontend aufsetzen	1	1
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	11	8.5
Erreichte Ziele	• Ich konnte die Phase Entscheiden abschliessen und mit der Phase Realisieren beginnen.		
Aufgetretene Probleme	• Bei der Datenbankmigration hatte ich Probleme mit den installierten Packages, da ich aus Versehen ein Virtual Environment mit Python 3.12 erstellt habe, aber das Skript mit Python 3.10 versucht habe zu starten. Aus diesem Grund hat es nicht alle installierten Packages erkannt. Nach genauerem Hinsehen konnte ich aber das Problem identifizieren und lösen.		
Tests	• Ich habe die Datenmigration laufen lassen und die migrierten Daten kurz überflogen. Sie sehen auf den ersten Blick korrekt aus. Ich werde sie genauer kontrollieren in der Phase "Kontrollieren"		
Hilfestellungen	• Ich nutzte ChatGPT, um meine Texte zu überarbeiten, verständlicher zu schreiben und auf Rechtschreibung zu prüfen. • Ich nutzte folgende Webseite, um genutzte Internetquellen korrekt zu kennzeichnen in meiner Dokumentation: https://www.scribbr.ch/richtig-zitieren-ch/internetquellen-zitieren/		
Vergleich Zeitplan	• Ich bin immer noch sehr gut im Zeitplan. Ich konnte einen Tag früher als geplant mit dem Realisieren beginnen.		
Positives	• Phase Entscheiden abgeschlossen • Phase Realisieren begonnen • Sehr gut im Zeitplan • Erfolgreiche Datenmigration		
Negatives	• Beim Start der Realisierungsphase habe ich ein wenig unsauber gearbeitet und ein Durcheinander mit den Python Versionen gemacht. Trotz allem konnte ich das Problem schnell beheben.		
Reflexion	• Grundsätzlich bin ich sehr zufrieden mit dem Tempo und der Qualität meiner heutigen Arbeit. Ich bin froh, dass ich mit dem Konzeptionieren fertig bin und mich jetzt an die Umsetzung setzen kann. Ich nehme von heute mit, dass ich bei der Umsetzung volle Konzentration zeigen muss und nicht zu viel Verschiedenes auf einmal machen will. So verhindere ich Unsauberkeiten und fördere die Qualität des Codes und des Endprodukts.		

Tabelle 11 Arbeitsjournal Tag 4

8.5 Arbeitsjournal Tag 5

Datum	Mittwoch, 23.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 8.5h	Soll	Ist
	• Backend und Frontend aufsetzen	1.5	1.5
	• Backend Endpoints implementieren	5.5	5.5
	• Bugfixing	0.5	0.5
	• Dokumentation überarbeiten	0.5	0.5
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	8.5	8.5
Erreichte Ziele	• Applikationen aufgesetzt und die API-Endpoints begonnen zu implementieren.		
Aufgetretene Probleme	• Zu Beginn Verbindungsprobleme mit der Datenbank vom Backend aus • Probleme mit pydantic-Response-Modell, da ich der Variabel created_at den falschen Typ zugewiesen habe. (str anstatt datetime)		
Tests	• Ich habe die implementierten Backend API-Endpoints manuell getestet und kontrolliert. Die Endpoints haben nicht von Anhieb funktioniert, weshalb ich Änderungen am Code machen musste, bis sie dann vollständig funktionierten.		
Hilfestellungen	• Offizielle FastAPI Doku: https://fastapi.tiangolo.com		
Vergleich Zeitplan	• Ich bin weiterhin sehr gut im Zeitplan. Ich konnte heute so viel erledigen wie geplant war und deshalb gibt es keine zeitlichen Veränderungen zu gestern.		
Positives	• Ich konnte die Applikationen zeitgemäss aufsetzen und mit der Implementierung der API-Endpoints beginnen, wo ich gut vorwärtsgekommen bin. • Bei Problemen konnte ich mir selbstständig und in angemessener Zeit helfen.		
Negatives	• Nichts, die aufgetretenen Probleme sehe ich nicht als negativ an, da dies zur Entwicklung dazugehört und ich genau für so Momente Bugfixing im Zeitplan eingetragen habe.		
Reflexion	• Ich bin froh, dass ich mehr zum Entwickeln komme und nicht nur Dokumentieren muss. Ich bin aber froh, dass ich die Konzepte im Vorhinein so gut dokumentiert habe. Das hilft mir beim Entwickeln. Ich nehme mit, dass eine präzise, vollständige Planung sehr wichtig ist, bevor man mit dem Realisierung Teil beginnt.		

Tabelle 12 Arbeitsjournal Tag 5

8.6 Arbeitsjournal Tag 6

Datum	Donnerstag, 24.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 9h	Soll	Ist
	• Backend Endpoints	3	3
	• Frontend implementieren	5	5
	• Bugfixing	0.5	0.5
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	9	9
Erreichte Ziele	• Funktionierende Endpoints im Backend • Frontend Implementation gestartet		
Aufgetretene Probleme	• Ich habe bei der Umsetzung des Frontends gemerkt, dass ich den Endpoint, um alle Cases abzurufen nicht korrekt implementiert habe im Backend. Ich habe den Status mit typ String implementiert. Dieser soll aber eigentlich ein Enum sein, welches entweder closed oder open ist.		
Tests	• Endpoints getestet		
Hilfestellungen	• Ich habe den Header und Footer von bestehenden Applikationen kopiert und diese dann dem Mockup entsprechend für dieses Projekt angepasst. Das weil SDX keine Header, bzw. Footer Komponenten anbietet.		
Vergleich Zeitplan	• Ich bin weiterhin gut im Zeitplan.		
Positives	• Endpoints funktionieren, wie erwartet. • Das Frontend sieht bereits vielversprechend aus und kommt voran.		
Negatives	• Aufgetretene Probleme		
Reflexion	• Ich habe den Endpoint im Backend so angepasst, dass er korrekt ist und werde morgen noch in der Doku die Anpassungen vornehmen. • Ich habe die Planung zu wenig genau angeschaut und mich zu schnell und unüberlegt ans Implementieren gesetzt. • Ich lerne daraus, die Planung möglichst genau zu studieren und die Applikation nach dieser umzusetzen. So kann ich die Arbeit effizienter gestalten und Fehler minimieren. • Ich lerne aber auch, dass es dazu gehört Fehler zu machen.		

Tabelle 13 Arbeitsjournal Tag 6

8.7 Arbeitsjournal Tag 7

Datum	Freitag, 25.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden	Soll	Ist
	• Frontend	3	5
	• Bugfixing	1	1
	• Backend	1	1
	• Dokumentation überarbeiten	0.5	0.5
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	5	8
Erreichte Ziele	• Ich konnte das Frontend fast vollständig umsetzen. • Backend funktioniert		
Aufgetretene Probleme	• Ich hatte Probleme mit dem ORM. Ich habe gemerkt, dass ich die Relationships zwischen den Tabellen noch nicht implementiert habe. Beim Implementieren dieser Relationships traten Probleme auf, da ich diese nicht korrekt konfigurierte.		
Tests	• API-Endpoints regelmässig über SwaggerUI Oberfläche getestet. • Frontend Funktionalitäten fortlaufend manuell getestet.		
Hilfestellungen	• FastAPI Dokumentation • Sqlalchemy Dokumentation • ChatGPT für Debugging Hilfe, indem ich mir die Error Message erklären lies.		
Vergleich Zeitplan	• Heute bin ich etwas weniger schnell vorwärtsgekommen, wie geplant. • Trotzdem bin ich immer noch sehr gut im Zeitplan.		
Positives	• Basic Frontend steht und Endpoints können von Frontend aus aufgerufen werden. • Daten werden korrekt wiedergegeben.		
Negatives	• Erstellung von Relationship models mit Sqlalchemy führte zu Problemen.		
Reflexion	• Ich konnte die Hindernisse eigenständig überwinden, indem ich im Internet gegrübelt habe und KI nach Hilfe gefragt habe. Ich nehme mit, dass man im Internet zu jedem Problem eine Lösung finden kann, wenn man richtig sucht und sich so Bestens selbst helfen kann. • Ich bin froh, dass ich die letzten Tage Arbeit vorholen konnte. Da ich heute nicht so produktiv war, bin ich dank diesem Vorsprung weiterhin gut im Zeitplan.		

Tabelle 14 Arbeitsjournal Tag 7

8.8 Arbeitsjournal Tag 8

Datum	Montag, 28.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 8.5h	Soll	Ist
	• Tests schreiben	4	4
	• Bugfixing	0	0.5
	• Testinfrastruktur	0.5	0.5
	• Arbeitsjournal	0.5	0.5
	• Expertenbesuch	1	1
	• Testdurchführung	2	2.5
	Total in Stunden	8	9
Erreichte Ziele	• Integrationstests geschrieben und dokumentiert		
Aufgetretene Probleme	• Die Datenbankstruktur war nicht korrekt. Part sollte nullable sein, war aber auf not null gesetzt. • Ich habe die nötigen Änderungen vorgenommen und dokumentiert.		
Tests	• Ich habe alle definierten Testfälle durchgeführt und dokumentiert		
Hilfestellungen	• Der HE hat mich darauf hingewiesen, dass ich das Testkonzept verfeinern und genauer dokumentieren muss.		
Vergleich Zeitplan	• Ich bin immer noch im Zeitplan. Trotzdem muss ich die nächsten zwei Tage noch Gas geben, um den Code zu fertigen und das Testkonzept auszuarbeiten.		
Positives	• Die meisten Tests sind erfolgreich durchlaufen		
Negatives	• Ich habe die Durchführung der Tests zu präzis dokumentiert. Diese Dokumentation hat mich mehr Zeit gekostet, als ich benötigt hätte.		
Reflexion	• Ich hätte die Tests früher mit meiner VF besprechen und präzisieren müssen. Ausserdem wäre so klar gewesen, dass ich das Testkonzept verfeinern und die Testdurchführung nicht so genau dokumentieren muss. Ich lerne daraus solche wichtigen Schritte, wie das Testkonzept von anderen frühzeitig zu überarbeiten.		

Tabelle 15 Arbeitsjournal Tag 8

8.9 Arbeitsjournal Tag 9

Datum	Dienstag, 29.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden: 9h	Soll	Ist
	• Backend API-Endpoint	0.5	0.5
	• Frontend	2.5	2.5
	• Testkonzept und Durchführung	2	4
	• Dokumentation	1	1.5
	• Arbeitsjournal	0.5	0.5
	Total in Stunden	6.5	9
Erreichte Ziele	• Applikation erfüllt alle Anforderungen und Tests		
Aufgetretene Probleme	• Ich habe bei der Pagination im Frontend ein Select implementiert, um das Limit zu wählen. Der gewählte Wert habe ich dann einem State mitgegeben. Das Problem war, dass dieser Wert kein Int war, sondern ein String. Es führte zu keiner Errormeldung, aber es führte zu Bugs bei der Anzeige und der Logik im Frontend. • Ich habe diesen String zu einem Int geparset, bevor ich ihn übergebe.		
Tests	• Ich konnte den Test T-06 auch erfolgreich testen, nachdem ich die Pagination korrekt implementiert habe.		
Hilfestellungen	• Ich nutze folgende Internetseite, um mir Internetseiten korrekt zu zitieren und mir die Arbeit mit dem Quellenverzeichnis zu vereinfachen: https://www.scribbr.ch/richtig-zitieren-ch/internetquellen-zitieren/		
Vergleich Zeitplan	• Ich habe morgen noch einen ganzen Tag Zeit, um die Reflexion und die Kurzfassung zu schreiben, sowie die Doku zu überarbeiten. Diese Zeit sollte mir reichen und die Arbeiten im Zeitplan werden somit perfekt ausgefüllt.		
Positives	• Applikation funktioniert und erfüllt alle Anforderungen. • Tests sind alle erfolgreich durchgeführt worden.		
Negatives	• Ich muss die Dokumentation noch ziemlich überarbeiten und auf Rechtschreibfehler prüfen. • Bei der Planung habe ich nicht bedacht, dass ich die Gesamtzahl der Cases übergeben muss und darum diesen API-Endpoint nicht geplant.		
Reflexion	• Heute konnte die letzten Schritte an der Applikation vornehmen, was mir ein gutes Gefühl gibt. Ich bin zufrieden mit meiner Leistung. Ich lerne, wie wichtig ein Testkonzept ist, um die Leistungen zu kontrollieren und nachzuweisen.		

Tabelle 16 Arbeitsjournal Tag 9

8.10 Arbeitsjournal Tag 10

Datum	Mittwoch, 30.04.2025		
Ausgeführte Arbeiten	Zeit in Stunden	Soll	Ist
	• Kurzfassung	0.5	0.5
	• Nacharbeiten definieren	0.5	0.5
	• Persönliches Fazit	2	2.5
	• Verbesserungen Dokumentation	0.5	2.5
	• Arbeitsjournal	0.5	0.5
	• Reserve	2	2
	Total in Stunden	6	8.5
Erreichte Ziele	• Dokumentation überarbeitet und abgeschlossen. • Die geplante Reserve habe ich für die Überarbeitung der Dokumentation verwendet • IPA fertiggestellt		
Aufgetretene Probleme	• Beim Erstellen der PDF-Datei sind die Kommentare ersichtlich gewesen. Ich musste diese ausschalten und nochmals generieren. • Ausserdem wurden gewisse Werte im Zeitplan als "#" dargestellt. Dies konnte ich lösen, indem ich die Spaltenbreite vergrösserte.		
Tests	• Ich habe frühzeitig am Morgen ein PDF aus der Word Dokumentation erstellt, um zu sehen, ob alles korrekt formatiert. • Ich habe schon vor der offiziellen Abgabe eine Version meiner IPA im PKORG hochgeladen.		
Hilfestellungen	• Ich habe die Dokumentation von Dritten durchlesen lassen. Das Dokument wurde mit Kommentaren über Verbesserungsvorschläge versehen. Anhand dieser Kommentare habe ich Textüberarbeitungen vorgenommen.		
Vergleich Zeitplan	• Ich konnte alle geplanten Arbeiten abschliessen. Auch wenn ich nicht alle Arbeiten am geplanten Tag erledigt habe.		
Positives	• Ich bin sehr zufrieden mit dem Endergebnis.		
Negatives	• Nichts		
Reflexion	• Ich bin froh, dass diese zehn anspruchsvollen Tage vorbei sind und die IPA abgeben konnte. Ich bin stolz darauf, dass ich alle Anforderungen abdecken konnte und bin zufrieden mit dem Resultat der IPA.		

Tabelle 17 Arbeitsjournal Tag 10

9 Persönliches Fazit

9.1 Herausforderungen

Die grösste Herausforderung im Projekt war das Zeitmanagement. Rückblickend bin ich sehr froh, dass ich mir zu Beginn einen strukturierten und übersichtlichen Zeitplan erstellt habe. Dieser half mir dabei, meinen täglichen Fortschritt zu beurteilen und rechtzeitig zu erkennen, wo ich zeitlich aufholen musste. Natürlich verlief nicht alles exakt nach Plan. Unerwartete technische Probleme gehörten zum Entwicklungsprozess dazu. Die Planungsphase habe ich etwas überschätzt und die Realisierungsphase, insbesondere das Frontend, unterschätzt. Ich nahm an, dass die Verwendung der SDX-Komponenten die Umsetzung beschleunigen würde, jedoch erforderte beispielsweise die Implementierung der Pagination deutlich mehr Zeit als gedacht.

9.2 Lernerfahrung

Während der IPA konnte ich besonders im Bereich der Projektplanung wertvolle Erfahrungen sammeln. Es wurde mir einmal mehr bewusst, wie entscheidend eine gute Planung ist – selbst bei einem Proof of Concept. Normalerweise bin ich stärker in der Umsetzung als in der Planung involviert. Daher war es für mich eine spannende und lehrreiche Erfahrung, ein Projekt von Grund auf selbst zu planen und umzusetzen. Ich habe meine Ziele über die gesamte IPA hinweg effektiv und selbstständig verfolgt. Regelmässige Meetings mit meiner VF halfen dabei, offene Fragen frühzeitig zu klären und Unklarheiten gezielt auszuräumen.

9.3 Entwicklungsperspektiven

Wenn ich das Projekt nochmals starten könnte, würde ich mir von Anfang an mehr Zeit für das Testkonzept einplanen und mit diesem auch früher starten. Das Testkonzept ist enorm wichtig, da man dadurch einen gewissen Leistungsnachweis erhält und ich meine Arbeit prüfen lassen kann – auch von Dritten.
Zudem würde ich den Zeitplan noch detaillierter strukturieren und grössere Aufgabenbereiche wie das Frontend in klar definierte Teilabschnitte wie „Detailansicht“ oder „Pagination-Funktion“ aufteilen. So liesse sich der Fortschritt noch präziser verfolgen.

9.4 Beurteilung des Erfolgs

Mit dem Projektverlauf und dem Resultat bin ich enorm zufrieden. Ich bin stolz darauf, dass ich die definierten Anforderungen und Ziele in der gegebenen Zeit erreicht habe. Es traten keine grösseren Probleme auf, was mir sicher entgegenkam. Kleinere Schwierigkeiten konnte ich eigenständig lösen. Besonders mein Engagement und meine Motivation zu Projektbeginn ermöglichten mir einen Vorsprung, den ich später nutzen konnte, um das Projekt erfolgreich abzuschliessen.

Teil 2: Projekt-Dokumentation

Swisscom (Schweiz) AG
Scheibler Benjamin
Förrlibuckstrasse 181
8005 Zürich
Mobile +41 79 841 95 00
E-Mail benjischeibler@gmail.com

10 Projektjournal

Im Projektjournal werden die Entscheidungen und Gespräche während der Facharbeit festgehalten.

10.1 Erster Expertenbesuch

Arbeitstag:	Dienstag, 15.04.2025
Ort	Remote
Beteiligte Personen	Marco Lupi, Livio Studach, Laura Lenhardt, Benjamin Scheibler
Inhalt	• Kennenlernen • Offene Fragen geklärt von beiden Seiten aus • Nächste Termine abgemacht
Beschlüsse	• Granularität auf 2h-Blöcke anpassen im Zeitplan • Nächster Besuch am 28.04.
Anmerkungen	Zeitplan und Arbeitsjournal bis morgen Abend an HE per Mail senden und auf PKORG hochladen.

Tabelle 18 Projektjournal erster Expertenbesuch

10.2 Besprechung Tag 1

Arbeitstag:	Dienstag, 15.04.2025
Ort	Zürich
Beteiligte Personen	Laura Lenhardt, Benjamin Scheibler
Inhalt	• Teil 1 der Dokumentation • Fragen Expertengespräch
Beschlüsse	• Verzeichnisse aktuell halten • Kurzfassung im Präteritum und Umsetzung mehr gewichten
Anmerkungen	Keine

Tabelle 19 Projektjournal Besprechung Tag 1

10.3 Besprechung Anforderungen

Arbeitstag:	Dienstag, 15.04.2025
Ort	Zürich
Beteiligte Personen	Laura Lenhardt, Benjamin Scheibler
Inhalt	• Anforderungen auf Verständlichkeit überprüft und validiert. • Relevanz überprüft.
Beschlüsse	• Anforderung F-02 unterteilt, da nicht alle Unterpunkte die gleiche Relevanz besitzen. • Formulierung für Anforderung F-04 verständlicher gestaltet.
Anmerkungen	Die Beschlüsse wurden umgesetzt, indem ich die Anforderungen dem Feedback entsprechend anpasste. Die Anforderungen wurden von allen relevanten Stakeholdern (Laura Lenhardt) abgenommen.

Tabelle 20 Besprechung Anforderungen

10.4 Besprechung Gestaltungsentwurf

Arbeitstag:	Dienstag, 22.04.2025
Ort	Zürich
Beteiligte Personen	Laura Lenhardt, Benjamin Scheibler
Inhalt	• Feedback zu den erstellten Mockups eingeholt

Beschlüsse	• Grundlegend gute Struktur.
	  ○ Übersichtlich und Userfreundlich gestaltet
	  ○ Den Anforderungen entsprechend
	• Typos korrigiert
	• Icons auf den Buttons weiss, nicht schwarz, da dies besser ist für die Accessibility und somit userfreundlicher
	• Anderes, verständlicheres Icon gewählt bei Close Case Button
	• Close Case Modal:
	  ○ Status als Pflichtfeld markieren.
	  ○ Close Case Button erst klickbar machen, wenn der User ein Status angewählt hat.

Anmerkungen	Funktionale Anpassung des Mockups wurden vorgenommen und die Dokumentation dementsprechend aktualisiert. Die Mockups im Kapitel 13.6 sind aktuell und angepasst nach den oben genannten Beschlüssen. Sie wurden nochmals besprochen und sind so offiziell von den Stakeholdern abgenommen.

10.5 Zweiter Expertenbesuch

Arbeitstag:	Montag, 28.04.2025
Ort	Zürich
Beteiligte Personen	Marco Lupi, Livio Studach, Laura Lenhardt, Benjamin Scheibler
Inhalt	• Applikation demonstriert • Testkonzept • Weiterer Ablauf besprochen • Unklarheiten bereinigt
Beschlüsse	• Klassifizierungsbaum in Querformat nehmen und auf zwei Seiten aufteilen. • Präsentation wird Monolog sein. • Demo wird Diskussion sein. • Archivierungsfunktion der Cases nach einer gewissen Spanne als Nacharbeit definiert. • Testkonzept überarbeiten und Erwartete Resultate präzisieren.
Anmerkungen	-

10.6 Besprechung Stand IPA

Arbeitstag:	Dienstag, 29.04.2025
Ort	Remote
Beteiligte Personen	Laura Lenhardt, Benjamin Scheibler
Inhalt	• Fortschritte überprüft • Testkonzept besprochen • Mockup überarbeitet • Letzte Schritte definiert

Beschlüsse	• Beim Mockup und Frontend habe ich zwei Buttons hinzugefügt, um bei der Case Übersichtsseite die Seiten zu wechseln. Das Mockup wurde dementsprechend angepasst und ist aktuell.
	• Wir haben ausserdem mögliche Nacharbeiten definiert
	  ○ Pagination für Incident und Matches und nicht nur für Cases.
	  ○ Filterung der Cases nach Datum

Anmerkungen	Weitere mögliche Nacharbeiten werden in Kapitel 17.2 definiert.

11 Kurzfassung

11.1 Ausgangslage

Der Sensitive Data Scanner (SDS) ist ein Tool, das dazu dient, sensitive Daten auf Dateisystemen automatisch zu identifizieren. Dieses Tool wird verschiedenen B2B Kunden angeboten, um deren Datenbestände auf potenzielle Datenschutzrisiken zu überprüfen.

Aktuell besitzt SDS jedoch keine Benutzeroberfläche. Die Ergebnisse eines Scans werden den Kunden ausschliesslich in Form eines Excel-Reports zur Verfügung gestellt. Dieser Report besteht aus mehreren, teils unübersichtlichen Tabellenblättern und ist für den Kunden nur schwer zu nutzen.

Das manuelle Arbeiten mit diesen Excel-Dateien ist ineffizient. Als Folge bleiben viele identifizierte Funde sensitiver Daten unbearbeitet. Dadurch werden sensible Daten nicht rechtzeitig gelöscht, verschlüsselt oder in sichere Speicherorte verschoben. Dies erhöht das Risiko von Daten Leaks.

11.2 Umsetzung

Im Rahmen der IPA wurde ein Proof of Concept (PoC) für eine Webapplikation entwickelt, welche die Verwaltung der SDS-Scanergebnisse erheblich vereinfacht. Ziel war es, eine moderne, benutzerfreundliche Lösung zu erstellen, mit der Kunden die Fundstellen effizienter bearbeiten können.

Für die Umsetzung wurde die Projektmethode IPERKA angewendet. Nach einer strukturierten Informations- und Planungsphase wurden in der Realisierungsphase drei zentrale Komponenten realisiert:

- Ein Migrationsskript zum Import der Scan-Daten in eine PostgreSQL-Datenbank.
- Ein FastAPI-Backend mit REST API-Endpoints zur Datenabfrage und Datenbearbeitung.
- Eine Benutzeroberfläche zur Darstellung und Verwaltung der Incidents, gestaltet nach definierten Designrichtlinien.

11.3 Ergebnis

Das entwickelte Proof of Concept erfüllt alle definierten Anforderungen. Es ermöglicht die Anzeige offener und geschlossener Cases, eine Detailansicht pro Case, sowie die Möglichkeit das Case zu schliessen. Durch diese Struktur wird der manuelle Aufwand reduziert und die Arbeit der Benutzer effizienter gestaltet.

Nach Abschluss der IPA ist vorgesehen, das Projekt weiterzuentwickeln und in die bestehende Produktlandschaft zu integrieren. Ziel ist es, die Anwendung produktiv einzuführen und den Kunden als effizientes Werkzeug zur Verwaltung sensitiver Daten zur Verfügung zu stellen.

12 Informieren

12.1 Anforderungen

Die Anforderungen wurden im Rahmen eines klar definierten Prozesses gemeinsam mit dem Stakeholder überprüft und validiert. Zu diesem Zweck habe ich ein Meeting organisiert, in dem wir die Anforderungen Punkt für Punkt durchgegangen sind. Ich habe bei jeder Anforderung sichergestellt, dass sie verständlich formuliert ist, ihre Relevanz erläutert und Feedback eingefordert.
Änderungen und Anpassungen wurden in Kapitel 10.3 dokumentiert. Die überarbeiteten Anforderungen spiegeln den Konsens aller Beteiligten wider und sind offiziell abgenommen.

12.1.1 Funktionale Anforderungen

Nr.	Beschreibung	Relevanz
F-01	Die Applikation extrahiert vollständig Daten aus einem JSON-Report und migriert diese strukturiert in eine PostgreSQL-Datenbank. Beispielhafte JSON-Daten und das zugehörige Datenbankschema sind zu dokumentieren.	
F-02	Die Applikation muss über eine REST API verfügen, welche die folgenden Funktionen bereitstellt:	
	1. Abruf offener Cases	
	2. Detailabfrage eines einzelnen Case	
	3. Abruf geschlossener Cases	
	4. Schliessen eines Case mit optionaler Begründung	
F-03	Das Frontend bietet eine übersichtliche und leicht verständliche Darstellung aller Cases an. Die Darstellung wird zuvor mit der Erstellung eines Mockups definiert und mit den Stakeholdern abgestimmt.	
F-04	Das Frontend muss die Detailansicht eines Cases anzeigen können und alle Matches den jeweiligen Kategorien zugeordnet darstellen. Pro match soll folgendes angezeigt werden: • Part • Line • Position	
F-05	Das Frontend muss zwischen offenen und geschlossenen Cases unterscheiden. Der Benutzer kann die Ansichten wechseln.	
F-06	Der User muss einen Case als „remediated“ markieren und dabei optional einen Kommentar hinterlegen können.	
F-07	Der User muss einen Case als „false-positive“ markieren und dabei optional einen Kommentar hinterlegen können.	
F-08	Die Übersicht der Cases muss paginiert dargestellt werden, um auch bei grossen Datenmengen eine gute Benutzerfreundlichkeit zu gewährleisten.	

Tabelle 21 Funktionale Anforderungen

12.1.2 Nicht-Funktionale Anforderungen

	Beschreibung	Relevanz
NF-01	Der Quellcode muss gemäss den PEP8-Konventionen sowie den internen Swisscom SDX-Richtlinien umgesetzt werden.	
NF-02	Interne Commit Guidelines müssen konsequent angewendet werden.	

Tabelle 22 Nicht-Funktionale Anforderungen

12.1.3 Legende Relevanz

Farbe	Relevanz	Erklärung
	Sehr hoch	Unverzichtbar, Applikation würde nicht funktionieren ohne die Anforderung.
	Hoch	Applikation würde funktionieren, hätte aber Grundfunktionen nicht.
	Mittel	Wichtig in gewissen Fällen – beispielsweise bei einer enorm grossen Menge an Daten.
	Niedrig	Applikation würde auch ohne diese Anforderung vollständig funktionieren

Tabelle 23 Legende Relevanz

12.2 Daten

12.2.1 Datenquellen

Auf meinem lokalen Gerät wurde der SDS-Scanner installiert. Für den Scan verwende ich einen speziell dafür vorgesehenen Ordner auf meinem lokalen Dateisystem, welcher "Dummy"-Daten enthält. Nach Abschluss des Scans erstellt SDS automatisch einen Report im JSON-Format, der im definierten Output-Ordner abgelegt wird. Dieser Report enthält umfangreiche Informationen über den Scan, sowie alle identifizierten Stellen, an denen sensitive Daten gefunden wurden. Ich werde die benötigten Daten identifizieren, in die Datenbank migrieren und mit diesen arbeiten.

12.2.2 Datenbankstruktur

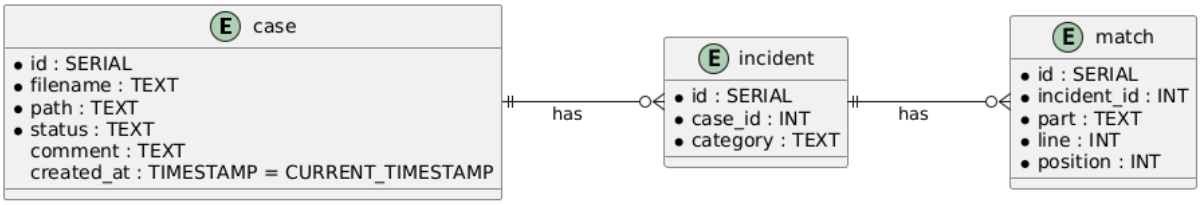


Abbildung 7 Datenbankstruktur

(PlantUML Web Server, n.d.)

Alle UML-Diagramme in dieser Dokumentation wurden mithilfe von PlantUML generiert.

12.2.3 4V-Modell

Volume (Menge): Beschreibt, wie viele Daten erzeugt, gespeichert und verarbeitet werden – also z. B. wie viele Cases oder Matches ein Scan generiert.

Velocity (Geschwindigkeit): Bezieht sich auf die Geschwindigkeit, mit der Daten entstehen, verändert oder verarbeitet werden.

Variety (Vielfalt): Umfasst die unterschiedlichen Formate, Strukturen und Arten der Daten.

Veracity (Zuverlässigkeit): Bewertet, wie genau und zuverlässig die Daten sind – insbesondere wichtig bei sensitiven Informationen, die false-positive sein können.

(Basra, 2024)

Table "case"

Spalte	Volume/ Menge	Velocity/ Geschwindigkeit	Variety/ Vielfalt	Veracity/ Zuverlässigkeit
Filename	Hoch	Niedrig	Mittel bis Hoch	Hoch
Path	Hoch	Niedrig	Niedrig	Hoch
Status	Hoch	Mittel	Niedrig	Niedrig

Tabelle 24 4V case

Table "incident"

Spalte	Volume/ Menge	Velocity/ Geschwindigkeit	Variety/ Vielfalt	Veracity/ Zuverlässigkeit
Case_id	Mittel	Niedrig	Niedrig	Hoch
Category	Mittel	Niedrig	Mittel	Mittel

Tabelle 25 4V incident

Table "match"

Spalte	Volume/ Menge	Velocity/ Geschwindigkeit	Variety/ Vielfalt	Veracity/ Zuverlässigkeit
Incident_id	Mittel	Niedrig	Niedrig	Hoch
Part	Mittel	Niedrig	Mittel	Mittel
Line	Mittel	Niedrig	Niedrig	Hoch
Position	Mittel	Niedrig	Niedrig	Hoch

Tabelle 26 4V match

Die Id Spalte jeder Tabelle wurde hier nicht bewertet, da sie Irrelevant ist. Nur die Id Spalte an sich ist nicht schützenswert.
Velocity ist stets niedrig, da wir in unserem Fall jeweils dasselbe Directory scannen und ein SDS-Scan meist periodisch läuft.s
Die einzelnen Werte an sich sind nicht schützenswert. Gefährlich wird es erst dann, wenn man Informationen von verschiedenen Tabellen hat. Wenn ich Part, Line und Position herausfinde, kann ich mit diesen Informationen nichts anfangen, sofern ich nicht weiss, zu welchem File diese Informationen gehören. Wenn ich aber Informationen von verschiedenen Tabellen habe, weiss ich, wo sich die sensitiven Daten befinden.

13 Planen

13.1 Softwarearchitektur

So könnte in die Architektur der Software aussehen.

Die API-Endpoints sind nur mal eine erste Idee. Man könnte beispielsweise auch ein PUT oder PATCH Request benutzen, um den Status des Cases zu ändern. Dies wird dann noch detaillierter im "Entscheiden" Teil behandelt.

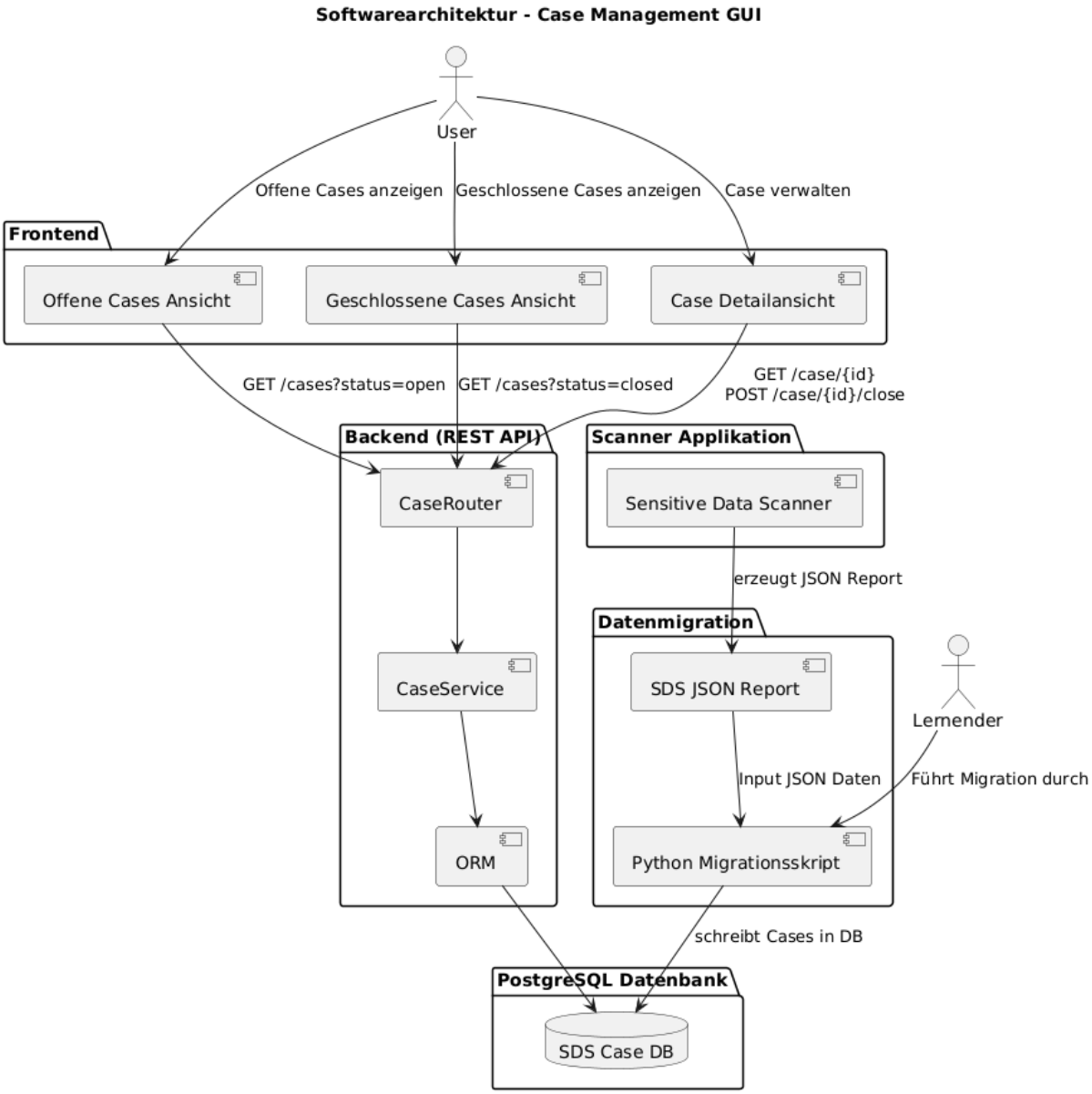


Abbildung 8 Softwarearchitektur

(PlantUML Web Server, n.d.)

13.2 Testkonzept

13.2.1 Testziele

Das Ziel der Tests ist es, alle technischen Anforderungen zu decken, zu prüfen und zu belegen.

13.2.2 Systemumgebung

- Backend: FastAPI Anwendung, gestartet auf http://localhost:8000
- Frontend: React-Anwendung, gestartet auf http://localhost:3000
- Datenbank: PostgreSQL, lokal installiert, befüllt mit Testdaten

Die Systemumgebung wird nach 15.2 Setup aufgesetzt.

13.2.3 Akteure

Akteur des Systems ist ein Benutzer mit vollen Rechten. Da das Login nicht Teil der IPA ist, besitzen alle Benutzer über volle Rechte und können alle Tests ausführen.

13.2.4 Daten

Die Tests werden mit Testdaten durchgeführt, welche in den einzelnen Testfällen genauer beschrieben werden. Alle verwendeten Daten entsprechen der Beschreibung von **12.2 Daten**.

13.2.5 Testinfrastruktur

Komponente	Beschreibung
Datenbank	PostgreSQL 17.4, lokal aufgesetzt mit folgender Anleitung Kapitel 15.2.1
Entwicklungsumgebung	Visual Studio Code, um Applikation zu starten und Tests zu schreiben, aufgesetzt nach: Kapitel 0
Backend	FastAPI-Anwendung, lokale gestartet und aufgesetzt nach: Kapitel 15.4.1
Frontend	React-Anwendung, lokal aufgesetzt und gestartet nach: Kapitel 15.5.1
Skripte	Python 3.10 Virtual Environment, Migrationsskript für Datenimport aus JSON, aufgesetzt nach:
Betriebssystem	Windows 11 Pro
Browser	Firefox Browser Version 137.0.2 für manuelle UI-Tests
API-Client	Postman Version 11.42.3 zur Durchführung API-Tests

Tabelle 27 Testinfrastruktur

13.2.6 Abdeckung Anforderungen

Test ID	Aktivität	Anforderung
T-01	Liste von Cases werden von API-Endpoint korrekt zurückgegeben	F-02.1, F-02.3
T-02	Details eines einzelnen Cases werden von API-Endpoint korrekt zurückgegeben	F-02.2
T-03	Auflistung der Cases wird korrekt dargestellt	F-03, F-05
T-04	Detailansicht des Case wird korrekt dargesetellt und zeigt korrekte Daten	F-04
T-05	Case kann mit einem Kommentar geschlossen werden	F-02.4, F-06, F-07
T-06	Applikation funktioniert bei grosser Datenmenge	F-08
T-07	Die migrierten Daten sind vollständig und korrekt	F-01

Tabelle 28 Abdeckung Anforderungen

13.2.7 Testfall T-01 Liste aller Cases API

Testszenario: Ein Benutzer möchte eine Übersicht aller offenen Cases anzeigen lassen via API.

Testvoraussetzung

- Die Testdatenbank enthält mindestens drei Cases:
  - Genau zwei mit Status open
  - Mindestens einer mit Status closed

```
UPDATE "case" SET status = 'remediated' WHERE status = 'open';
UPDATE "case" SET status = 'open' WHERE "case".id = 1 or "case".id =
2;
```

- API und Datenbank sind korrekt verbunden.
- Der Server ist gestartet und erreichbar:

```
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to
quit)
```

Testschritte

- GET-Anfrage an den Endpoint: http://localhost:8000/cases?status=open
- HTTP-Statuscode prüfen
- Struktur Prüfen der JSON-Response
  - Response ist ein Array.
  - Jedes Objekt enthält Felder id, status, filename und path.
- Verifizieren, dass nur Cases mit Status open enthalten sind.
- Anzahl zurückgegebener Cases zählen.

Testart: Integrationstest

Erwartete Resultate

- Die Antwort liefert HTTP-Status 200 OK.
- Struktur der JSON-Response ist korrekt und entspricht diesem Format:

```
  {
    "id": int,
    "filename": "string",
    "path": "string",
    "status": open,
    "comment": "string",
    "created_at": "datetime"
  }
```

- Die Antwort enthält exakt zwei Cases.

13.2.8 Testfall T-02 Detailansicht Case API

Testszenario: Ein Benutzer möchte die Detailansicht eines bestimmten Cases einsehen via API.

Testvoraussetzung

- Die Testdatenbank enthält mindestens ein Case mit einer bekannten ID, z.B. id=2:

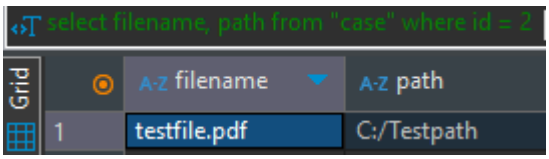


Abbildung 9 T-02 Testvoraussetzung

- API und Datenbank sind korrekt verbunden.
- Der Server ist gestartet und erreichbar:

```
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Testschritte

- GET-Anfrage an den Endpoint:
  GET http://localhost:8000/cases/2
- HTTP-Statuscode prüfen
- Struktur Prüfen der JSON-Response
  - Response ist ein JSON-Objekt.
  - Das Objekt enthält Felder: id, status, filename, path, incidents Array
  - Incidents ist ein Array mit Felder: category und matches Array
  - Matches enthält Felder wie part, line, position.
- Verifizieren, dass die ID des zurückgegebenen Cases 2 entspricht.
- Verifizieren, dass die Felder filename und path mit den Testdaten übereinstimmen.
- Verifizieren, dass das Feld category in incident und die dazugehörenden Matches den Testdaten entsprechen.

Testart: Integrationstest

Erwartete Resultate

- Die Antwort liefert HTTP-Status 200 OK.
- Das JSON enthält die vollständigen Details zum Case mit ID 2.
- Struktur entspricht folgendem JSON:

```
{
  "id": 2,
  "filename": "string",
  "path": "string",
  "status": "string",
  "comment": "string",
  "created_at": "datetime",
  "incidents": [
    {
      "category": "string",
      "matches": [
        {
          "part": "string",
          "line": 0,
          "position": 0
        }
      ]
    }
  ]
}
```

13.2.9 Testfall T-03 Auflistung Cases Frontend

Testszenario: Ein Benutzer möchte eine Übersicht aller offenen Cases anzeigen lassen.

Testvoraussetzung

- Die Testdatenbank enthält mindestens drei Cases:
  - Genau zwei mit Status open
  - Mindestens eines mit Status closed

```
UPDATE "case" SET status = 'remediated' WHERE status = 'open';
UPDATE "case" SET status = 'open' WHERE "case".id = 1 or "case".id =
2;
```

- Das Backend ist lauffähig und korrekt angebunden.
- Das Frontend ist gestartet und erreichbar über den Browser.

Testschritte

- User öffnet http://localhost:3000.
- Im Browser wird die Liste der offenen Cases geladen.
- Darstellung der Cases-Tabelle beobachten.

Testart: Systemtest

Erwartete Resultate

- Zwei Cases mit Status open werden auf der Seite angezeigt.
- Jedes Case enthält die korrekten Informationen gemäss den Testdaten.
- Keine geschlossenen Cases sind sichtbar.
- Jeder Case-Eintrag ist klickbar und führt zur Detailansicht.
- Das Layout ist korrekt und entspricht dem des Mockups.

13.2.10 Testfall T-04 Detailansicht Case Frontend

Testszenario: Ein Benutzer möchte die Detailansicht eines bestimmten Cases einsehen.

Testvoraussetzung

- Die Testdatenbank enthält ein Case mit bekannten Daten, z.B.
  - id=123
  - filename=file.pdf
  - path=C:/data/path/
  - status=open
  - Incident mit Kategorie "IBAN" mit zwei Matches

```
INSERT INTO "case" (id, filename, path, status)
VALUES (123, 'file.pdf', 'C:/data/path/', 'open');

INSERT INTO incident (id, case_id, category)
VALUES (1, 123, 'IBAN')

INSERT INTO match (id, incident_id, part, line, position)
VALUES
(1, 1, 'part1', 10, 5),
(2, 1, 'part2', 20, 15);
```

- Das Backend ist lauffähig und korrekt angebunden.
- Das Frontend ist gestartet und erreichbar über den Browser.
- Der User befindet sich auf der Übersichtsseite der Cases im Frontend (http://localhost:3000).

Testschritte

- User klickt auf gewünschtes Case mit id=123.
- Überprüfen, ob Informationen korrekt sind.

Testart: Systemtest

Erwartete Resultate

- Die Detailansicht (http://localhost:3000/cases/123) öffnet sich nach Klick auf ein Case
- Alle relevanten Daten des Cases werden vollständig und korrekt angezeigt und entsprechen den zuvor hinzugefügten Daten.
- Kategorien und deren Matches sind korrekt gruppiert und enthalten die zuvor hinzugefügten Werte.
- Das Layout entspricht dem des Mockups.

13.2.11 Testfall T-05 Case mit Kommentar schliessen

Testszenario: Ein Benutzer möchte ein offenes Case schliessen und gleichzeitig ein Kommentar hinterlassen.

Testvoraussetzung

- Die Testdatenbank enthält mindestens einen Case mit Status open.

Abbildung 10 T-05 Testvoraussetzung

- Die Applikation ist gestartet und im Browser erreichbar.
- Der Benutzer befindet sich in der Detailansicht eines offenen Cases (http://localhost:3000/cases/123).
- Ein Button zum Schliessen des Cases ist vorhanden.

Testschritte

- User klickt "Close case" Button.
- Modal öffnet sich.
- User wählt einen Status, entweder "false-positive" oder "remediated", und gibt einen Kommentar ein.
- User klickt auf Submit Button.

Testart: Systemtest

Erwartete Resultate

- Der Case wird geschlossen.
- Der Case ist nicht mehr auf der Übersichtsseite der geöffneten Cases ersichtlich.
- Der Case ist auf der Übersichtsseite der geschlossenen Cases zu sehen.
- Der Kommentar ist ersichtlich und entspricht dem zuvor eingegebenen Kommentar.

13.2.12 Testfall T-06 Funktionsfähigkeit mit grosser Datenmenge

Testszenario: Ein Benutzer möchte eine funktionsfähige Applikation, auch wenn die Datenbank viele Daten hat.

Testvoraussetzung

- Die Datenbank ist mit einer grossen Anzahl an Cases befüllt (>1000 Cases).
- Frontend und Backend sind gestartet und vollständig integriert.
- Pagination ist im Frontend, sowie Backend implementiert.

Testschritte

- Start der Applikation und Aufrufen der Startseite über http://localhost:3000.
- Warten, bis die Übersichtsseite der offenen Cases geladen ist.
- Beobachten und Messen der Ladezeit der ersten Seite.
- Überprüfen, ob die Pagination-Elemente korrekt gemäss Mockup dargestellt werden.
- Wechseln zur nächsten Seite über die Pagination-Steuerung.
- Bewerten, ob die Ladezeit und Reaktionszeit weiterhin akzeptabel bleiben.

Testart: Performancetest

Erwartete Resultate

- Die Startseite wird ohne spürbare Verzögerung geladen (innerhalb von zwei Sekunden).
- Pro Seite wird korrekt die ausgewählte Anzahl an Cases angezeigt.
- Die Pagination-Elemente erscheinen gemäss Mockup.
- Das Navigieren zwischen den Seiten funktioniert schnell und fehlerfrei.
- Kein Einfrieren und keine Abstürze beim Blättern oder Scrollen.

13.2.13Testfall T-07 Korrektheit migrierter Daten

Testszenario: Eine Datenbankmigration mit einem SDS-Scan Report wird durchgeführt.

Testvoraussetzung

- Ein JSON-Export des SDS-Scans mit bekannten Inhalten liegt vor.
- Das Migrationsskript ist fertiggestellt und einsatzbereit: Kapitel 0
- Die Datenbank ist leer oder vorbereitet für die Migration:

```
delete from match;
delete from incident;
delete from "case";
```

- Die Zielstruktur in der Datenbank ist korrekt eingerichtet: 15.2.1 Datenbank

Testschritte

- Das Migrationsskript wird mit dem JSON-File ausgeführt.
- Nach der Ausführung wird die Datenbankstruktur überprüft.
- Die Anzahl der migrierten Cases wird mit der Anzahl im JSON verglichen.
- Einzelne Datenpunkte (z. B. ein spezifischer Incident) werden stichprobenartig geprüft.
- Es wird festgestellt, ob alle Felder korrekt übernommen wurden:
  - Path, Filename, Incidents, Matches
- Die Relationen zwischen den Tabellen werden geprüft.
- Keine fehlerhaften oder unvollständigen Datensätze in der Datenbank werden gefunden.

Testart: Funktionstest

Erwartete Resultate

- Die Anzahl der Cases, Incidents und Matches in der Datenbank entspricht exakt der des JSON.
- Alle relevanten Felder wurden korrekt übernommen.
- Die Beziehungen zwischen den Tabellen sind korrekt.
- Es existieren keine fehlerhaften, leeren oder unvollständigen Einträge.

13.3 Migrationskonzept

13.3.1 Ziel

Das Ziel der Migration ist es alle relevanten Daten für dieses Projekt aus dem JSON herauszulesen und in die zuvor erstellte PostgreSQL Datenbank zu migrieren. Die migrierten Daten werden überprüft und validiert. Anschliessend, kann das Backend implementiert werden und API Requests auf die Datenbank ausgeführt werden.

13.3.2 JSON-Report

Der JSON-Report, der von SDS generiert wurde, hat folgende Struktur:

```
{
  "Matches": [
    {
      "S": "1",
      "BranchName": "",
      "Path": "C:/Users/taascbej/SDS_temp_scan",
      "FileName": "Lorem1.pdf",
      "ROWID": null,
      "Part": " Page 5",
      "Line": "38",
      "Position": "10",
      "Fuzzy": "1",
      "Length": 13,
      "Distance": null,
      "NotCounted": null,
      "Type": null,
      "Category": "PrivateNames",
      "Whitelist": null,
      "EDM": ".",
      "EDM_waived": null,
      "MLlabel": null,
      "MLconfidence": null,
      "PartnerID": ""
    },
      …
  ],
  "Breakdown": […],
  "Result_Files": […],
  "Result_Paths": […],
  "Exceptions": […]
}
```

Was im Fall dieser IPA relevant ist, sind die Daten, die sich im Array "Matches" befinden.
In den weiteren Arrays, Breakdown, Result_Files, Result_Paths und Exceptions sind Daten, die für dieses Projekt und dessen Anforderungen irrelevant sind. Das sind Informationen Scantyp, Textgrösse vom File oder den letzten Zugriff auf das File.

Auch im Matches Array sind nicht alle Daten relevant. Um besser zu verstehen, welche Daten relevant sind, werde ich eine Zuweisung der Daten machen.

13.3.3 Zuweisung JSON-Daten zur Datenbankstruktur

Erklärung:
Table <Tabellenname>
Tabellenspalte = "<dazugehöriger Key im JSON>"

Table case
id = "S",
filename = "Filename"
path = "Path"

Table incident
case_id = "S",
category = "Category"

Table matches
part = "Part",
line = "Line",
position = "Position"

13.3.4 Mögliche Probleme

Jeder Case wird über die Id im Feld S eindeutig identifiziert. Für jede Id darf nur einmal ein Datensatz in der Case-Tabelle erstellt werden. Alle weiteren zugehörigen Informationen, wie Incidents, müssen nur noch auf diese Id verweisen, wenn der Case schon erstellt wurde.

In der Incident Tabelle darf es pro Kombination aus case_id und category nur einen Eintrag geben. Da ein Case mehrere Matches derselben Kategorie enthalten kann, muss beim Einfügen sichergestellt werden, dass nicht mehrfach dieselbe Kombination eingefügt wird.

Line und Position sind in der JSON-Datei Strings. In der Datenbank sind sie aber vom Typ Int. Diese Werte müssen während der Migration korrekt umgewandelt werden, um Fehler bei der Datenbankeinfügung zu vermeiden.

13.3.5 Optionen

Es gibt verschiedene Optionen, wie man Daten aus einem JSON in eine PostgreSQL Datenbank migrieren kann, z.B.

- Migration mit einem ETL-Tool
- Migration durch ein Python-Skript mit psycopg

In der Phase "Entscheiden" werde ich genauer auf die Möglichkeiten eingehen und eine Entscheidung treffen.

13.4 Datenschutz

13.4.1 Klassifizierungsmodell

Innerhalb der Swisscom wird konsequent ein eigenes Klassifizierungsmodell angewendet. Die Aufgabe dieses Modells ist es, die Daten in ihrer entsprechenden Wichtigkeit und Schutzwürdigkeit einzustufen.

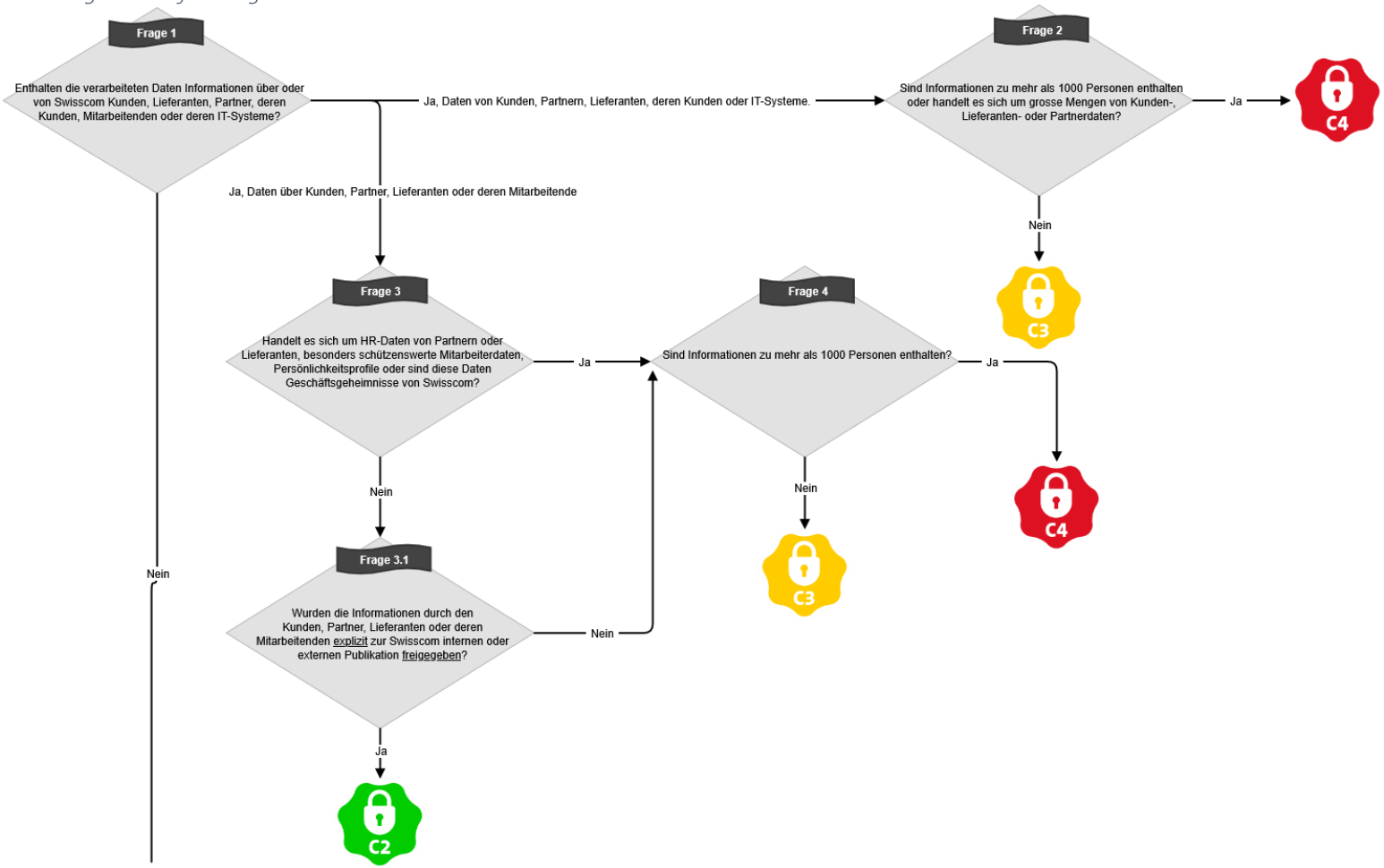
- C1: Diese Daten sind der gesamten Öffentlichkeit und internen Mitarbeitern zugänglich.
- C2: Diese Daten sind ausschliesslich für interne Mitarbeiter der Swisscom zugänglich.
- C3: Diese Daten sind von hoher Wichtigkeit und können nur von selektierten Gruppen oder Personen innerhalb von Swisscom eingesehen werden.
- C4: Diese Daten sind von immens hoher Wichtigkeit und können nur von spezifischen Personen eingesehen werden. Es handelt sich bei dieser Stufe meist um Daten von Banken.

In meiner IPA werde ich dieses auch verwenden und meine Daten mithilfe dieses Modells klassifizieren.
Ich beschreibe und klassifiziere mit diesem Modell bereits in der Planungsphase, sodass ich die Grundsätze von "Privacy By Design" verfolge und dieses Modell im gesamten Entwicklungsprozess miteinbeziehen kann.

Der Klassifizierungsbaum und die Klassifizierungstabelle in den folgenden zwei Kapiteln wurden aus interner Quelle bezogen. Anhand dieser werde ich die Klassifizierung meiner Daten durchführen.

13.4.2 Klassifizierungsbaum

Abbildung 11 Klassifizierungsbaum 1



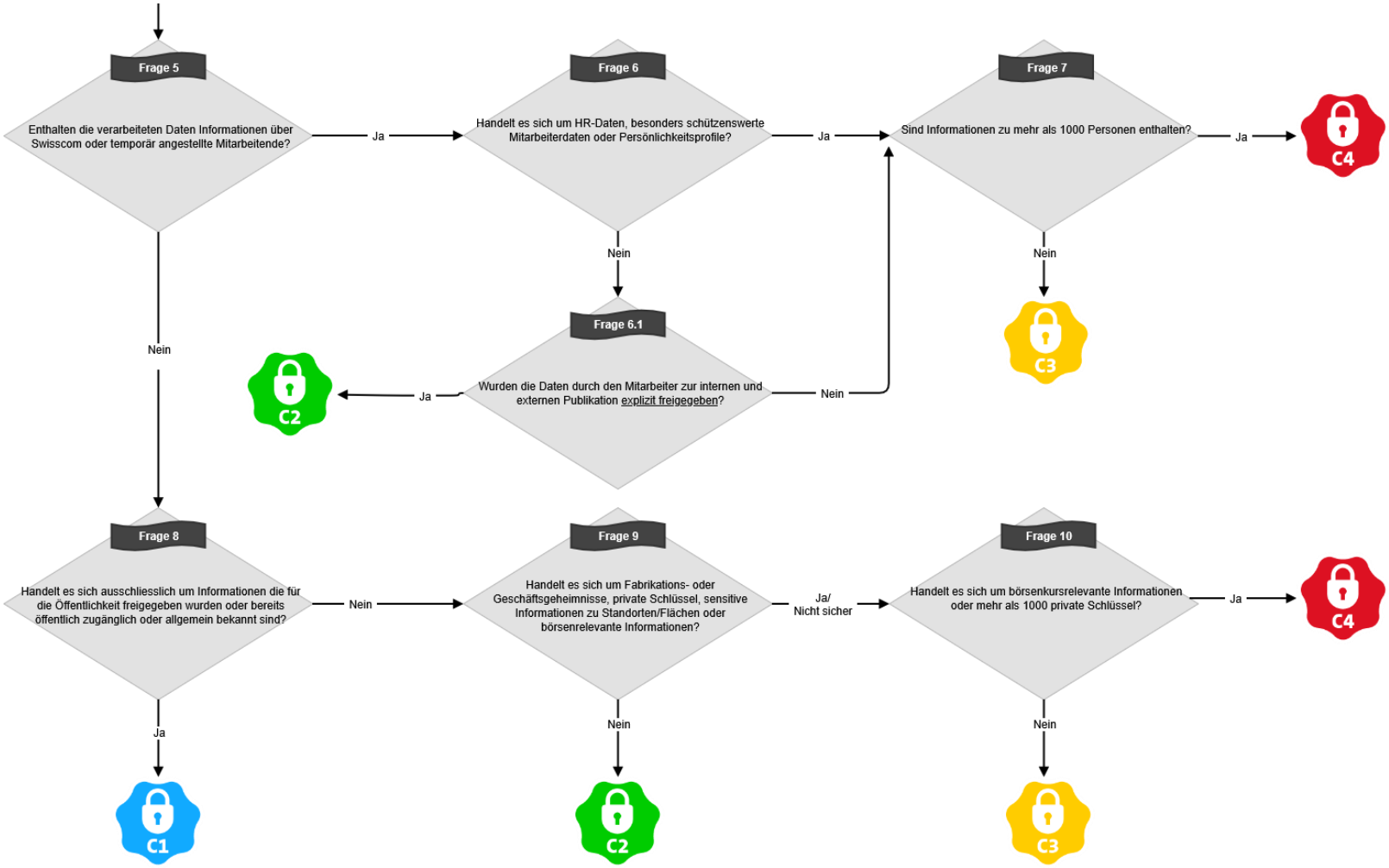


Abbildung 12 Klassifizierungsbaum 2

13.4.3 Klassifizierungstabelle

Frage Nr.	Titel	Frage	Antworten und nächste Frage
1	Kunden-/Partner-/Lieferantendaten	Enthalten die verarbeiteten Daten oder Daten in transit Informationen über oder von Swisscom Kunden, Lieferanten, Partner, deren Kunden, Mitarbeitenden oder deren IT-Systeme?	Ja, Daten von Kunden, Partnern, Lieferanten, deren Kunden oder IT-Systeme → Frage 2 Ja, Daten über Kunden, Partner, Lieferanten oder deren Mitarbeitende → Frage 3 Nein → Frage 5
2	Massendatenhaltung	Sind Informationen zu mehr als 1000 Personen enthalten oder handelt es sich um grosse Mengen von Kunden-, Lieferanten- oder Partnerdaten?	Ja → C4 Nein → C3
3	Sensitive Personendaten	Handelt es sich um HR-Daten von Partnern oder Lieferanten, besonders schützenswerte Mitarbeiterdaten, Persönlichkeitsprofile oder sind diese Daten Geschäftsgeheimnisse von Swisscom?	Ja → Frage 4 Nein → Frage 3.1
3.1	Freigegebene sensitive Personendaten	Wurden die Informationen durch den Kunden, Partner, Lieferanten oder deren Mitarbeitenden explizit zur Swisscom internen oder externen Publikation freigegeben?	Ja → C2 Nein → Frage 4
4	Massendatenhaltung	Sind Informationen zu mehr als 1000 Personen enthalten?	Ja → C4 Nein → C3
5	Mitarbeiterdaten	Enthalten die verarbeiteten Daten Informationen über Swisscom oder temporär angestellte Mitarbeitende?	Ja → Frage 6 Nein → Frage 8
6	Sensitive Mitarbeiterdaten	Handelt es sich um HR-Daten, besonders schützenswerte Mitarbeiterdaten oder Persönlichkeitsprofile?	Ja → Frage 7 Nein → Frage 6.1
6.1	Freigegebene sensitive Mitarbeiterdaten	Wurden die Daten durch den Mitarbeiter zur internen und externen Publikation explizit freigegeben?	Ja → C2 Nein → Frage 7
7	Massendatenhaltung	Sind Informationen zu mehr als 1000 Personen enthalten?	Ja → C4 Nein → C3

8	Öffentliche Daten	Handelt es sich ausschliesslich um Informationen, die für die Öffentlichkeit freigegeben wurden oder bereits öffentlich zugänglich oder allgemein bekannt sind?	Ja → C1 Nein → Frage 9
9	Fabrikations-/Geschäftsgeheimnisse	Handelt es sich um Fabrikations- oder Geschäftsgeheimnisse, sicherheitsrelevante/kritische Informationen, sensitive Informationen zu Standorten/Flächen oder börsenrelevante Informationen?	Ja oder unsicher → Frage 10 Nein → C2
10	Börsenkursrelevante Infos / Schlüssel	Handelt es sich um börsenkursrelevante Informationen oder mehr als 1000 private Schlüssel?	Ja → C4 Nein → C3

Tabelle 29 Klassifizierungstabelle

(Swisscom Wiki, 2021)

13.4.4 Klassifizierung Daten Tabellenebene

Tabelle	Klassifizierung	Weg im Entscheidungsbaum
Case	C4	1 → 2 → C4
Incident	C2	1 → 5 → 8 → 9 → C2
Match	C2	1 → 5 → 8 → 9 → C2

Tabelle 30 Klassifizierung Tabellenebene

13.4.5 Klassifizierung Daten Spaltenebene

Tabelle "case"

Spalte	Klassifizierung	Weg im Entscheidungsbaum
Id	C2	1 → 5 → 8 → 9 → C2
Filename	C4	1 → 2 → C4
Path	C4	1 → 2 → C4

Tabelle 31 Klassifizierung Tabelle case

Tabelle "incident"

Spalte	Klassifizierung	Weg im Entscheidungsbaum
Case_id	C2	1 → 5 → 8 → 9 → C2
Category	C2	1 → 5 → 8 → 9 → C2

Tabelle 32 Klassifizierung Tabelle incident

Tabelle "match"

Spalte	Klassifizierung	Weg im Entscheidungsbaum
Part	C2	1 → 5 → 8 → 9 → C2
Line	C2	1 → 5 → 8 → 9 → C2
Position	C2	1 → 5 → 8 → 9 → C2

Tabelle 33 Klassifizierung Tabelle match

13.5 Rollenkonzept

Das Rollenkonzept wird in der IPA nicht technisch umgesetzt, da dies ein Login benötigen würde und so den zeitlichen Rahmen dieser zehntägigen Arbeit überschreiten würde.
Teil der IPA ist aber die Beschreibung eines einsetzbaren Rollenkonzept, welches die Zuweisungen von Verantwortlichkeiten und Zugriffsrechte innerhalb des Systems festlegt.

13.5.1 Rollenbeschreibung

Rolle	Verantwortlichkeiten	Zugriffsrechte im System
Viewer	Einsehen von Cases	Einsicht in alle offenen/geschlossenen Cases Zugriff auf Detailansicht
Editor	Verarbeitung von Cases	Alle Viewer-Rechte Case schliessen
Administrator	Verwaltung der gesamten Applikation	Alle Rechte der Editor Datenmigration durchführen Systemparameter ändern (z. B. Kategorien)

Tabelle 34 Rollenbeschreibung

13.5.2 Zugriffskonzept

Komponente	Viewer	Bearbeiter	Admin
Übersicht offene Cases	Ja	Ja	Ja
Übersicht geschlossene Cases	Ja	Ja	Ja
Detailansicht Case	Ja	Ja	Ja
Case schliessen	Nein	Ja	Ja
Systemkonfiguration ändern	Nein	Nein	Ja
Datenimport/Migration	Nein	Nein	Ja

Tabelle 35 Zugriffskonzept

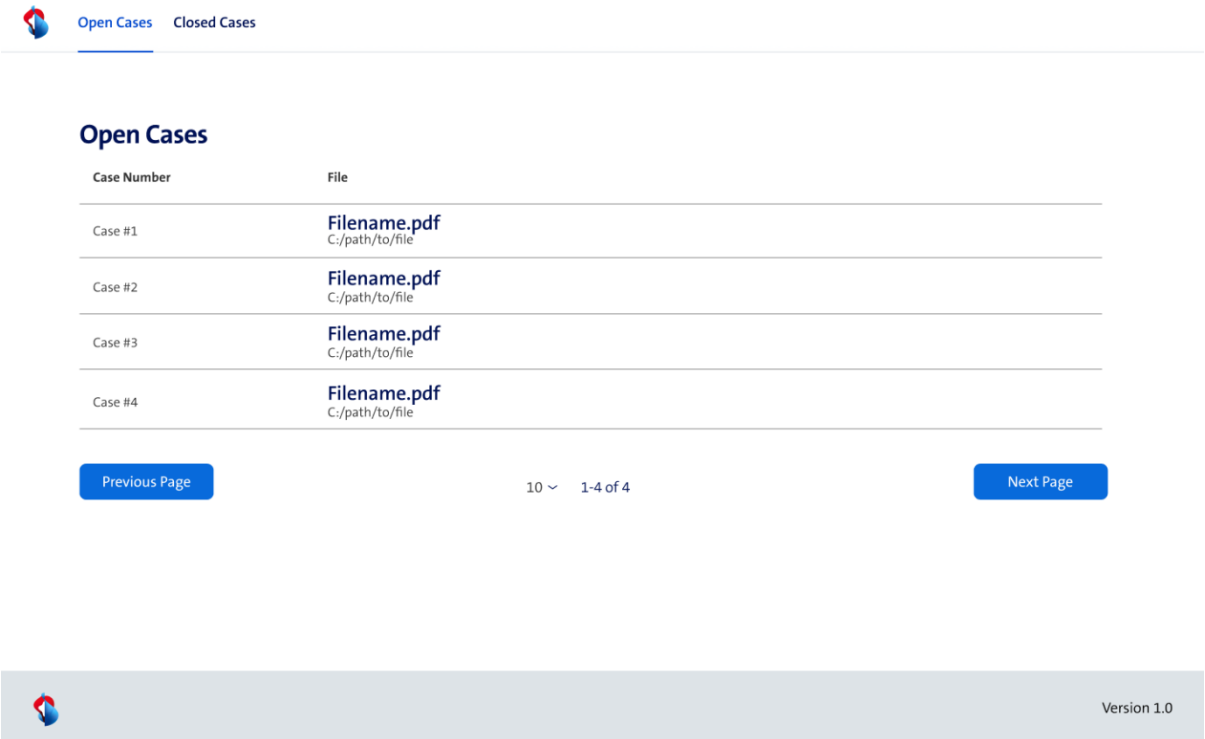
13.6 Gestaltungsentwurf

Der Gestaltungsentwurf (Mockup) wurde mithilfe des Tools "Figma" erstellt. Für dieses Tool habe ich als Swisscom Mitarbeiter eine gültige Lizenz und kann alle Funktionen und Werkzeuge des Tools nutzen. Dieses Tool eignet sich besonders gut zur Erstellung von Mockups für Benutzeroberflächen, die den Gestaltungsrichtlinien von Swisscom SDX entsprechen. Swisscom stellt in Figma eine eigene SDX-Komponentenbibliothek bereit, welche sämtliche UI-Elemente von SDX beinhaltet.
Diese Komponenten konnten direkt übernommen und an die spezifischen Anforderungen angepasst werden. Dadurch liess sich das Mockup effizient und den Richtlinien entsprechend umsetzen.
Das Feedback von den Stakeholdern wurde in einem Meeting eingeholt und in die Überarbeitung des Entwurfs integriert.
Das Feedback und die Entschlüsse wurden im Projektjournal im Kapitel 10.4 festgehalten und dokumentiert.

13.6.1 Übersichtsseite offene Cases

Die Übersichtsseite der offenen Cases zeigt alle Cases. Die Cases werden paginiert dargestellt, wobei es unten die Gesamtanzahl und die Anzahl dargestellter Cases anzeigt.

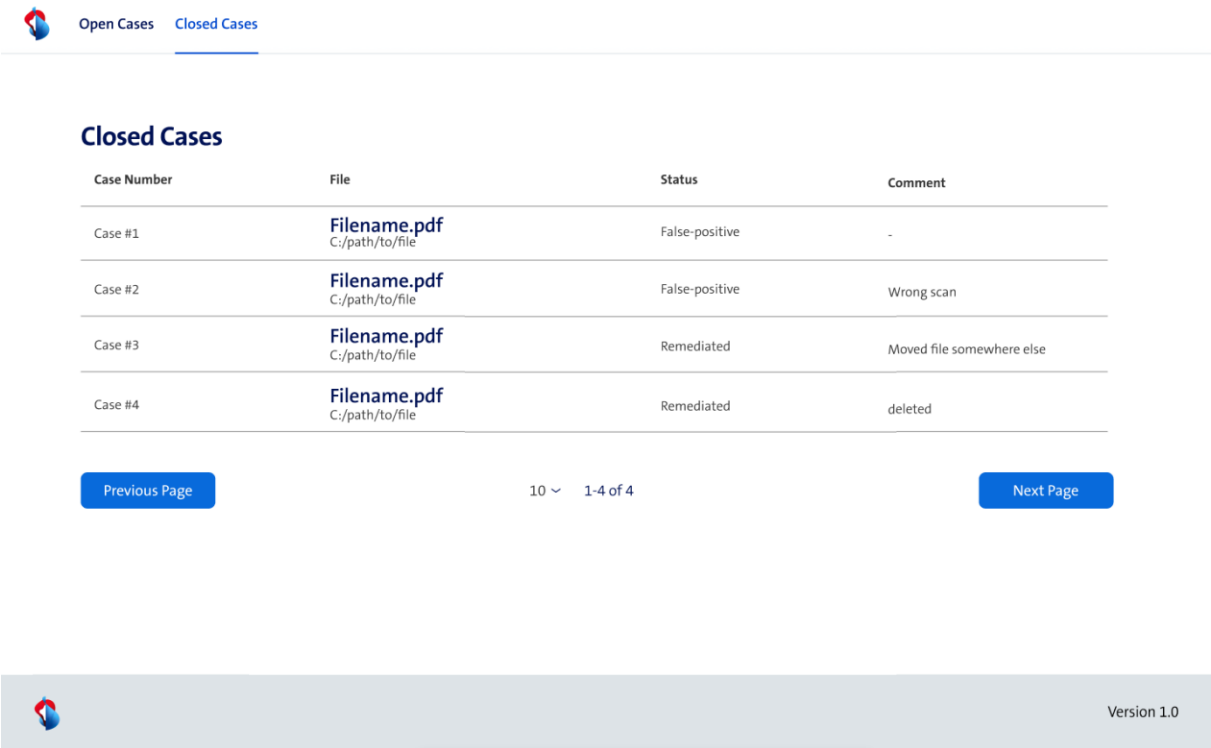
Diese Ansicht deckt folgende Anforderungen ab: F-03, F-05, F-08, NF-01



13.6.2 Übersichtsseite geschlossene Cases

Die Übersichtsseite der geschlossenen Cases zeigt zusätzlich noch den Status an, da dieser "false-positive" oder "remediated" sein kann bei den geschlossenen Cases. Ausserdem wird der dazugehörige Kommentar angezeigt, falls einer existiert. Auch hier werden die Cases paginiert dargestellt.

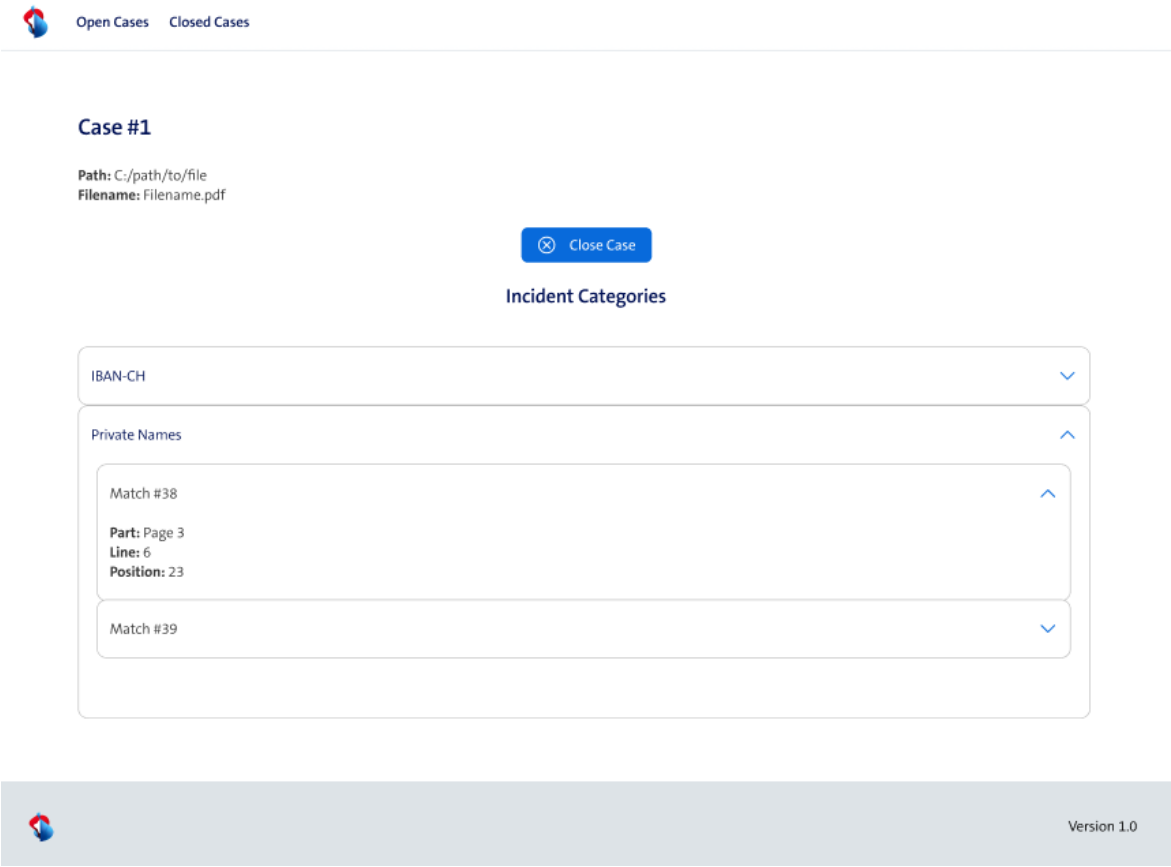
Diese Ansicht deckt folgende Anforderungen ab: F-03, F-05, F-08, NF-01



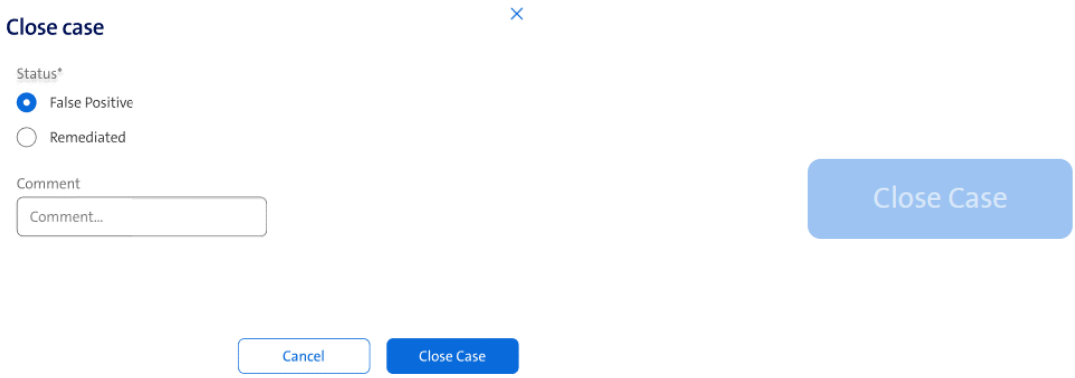
13.6.3 Detailansicht

In der Detailansicht sieht man die verschiedenen Kategorien, welche man einzeln aufklappen kann. Pro Kategorie kann man die dazugehörigen Matches aufklappen und den Fundort der sensitiven Daten ausfindig machen.

Diese Ansichten decken folgende Anforderungen ab: F-04, F-06, F-07



Wenn der Button "Close Case" gedrückt wird, erscheint folgendes Modal auf dem Bildschirm des Users. Wenn kein Status selektiert ist, ist der Button "Close Case" nicht klickbar, wie auf dem rechten Bild erkennbar:



13.7 Realisierungskonzept

13.7.1 Use Cases und Abläufe

Offene Cases anzeigen

Ein Benutzer möchte eine Übersicht aller offenen Cases einsehen, um den aktuellen Stand der offenen Fälle zu überprüfen.

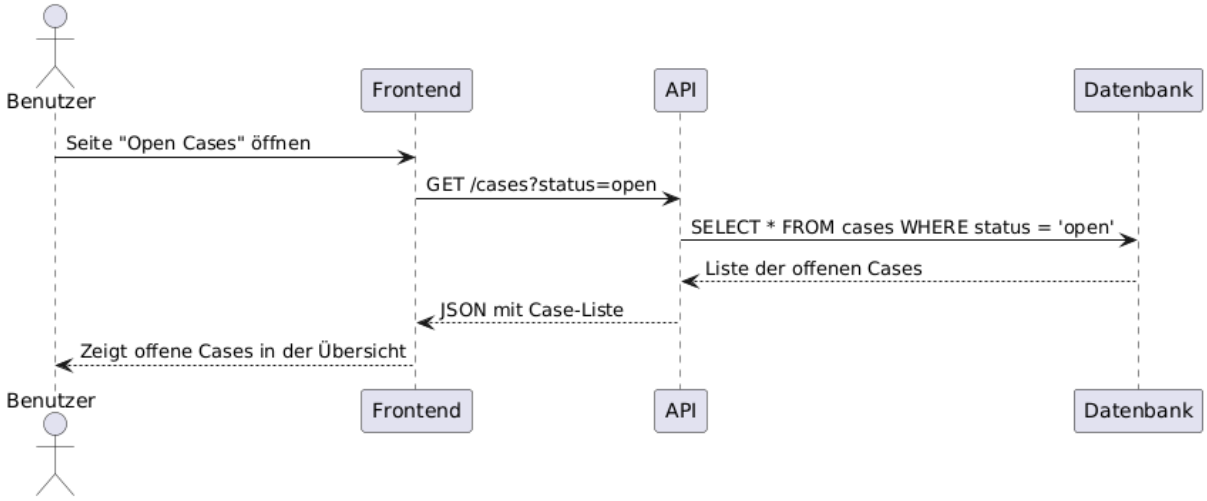


Abbildung 13 Sequenzdiagramm offene Cases anzeigen

(PlantUML Web Server, n.d.)

Geschlossene Cases anzeigen

Ein Benutzer möchte eine Übersicht von allen Cases, die er bereits geschlossen hat, um zu sehen, welche Files er bereits abgearbeitet hat.

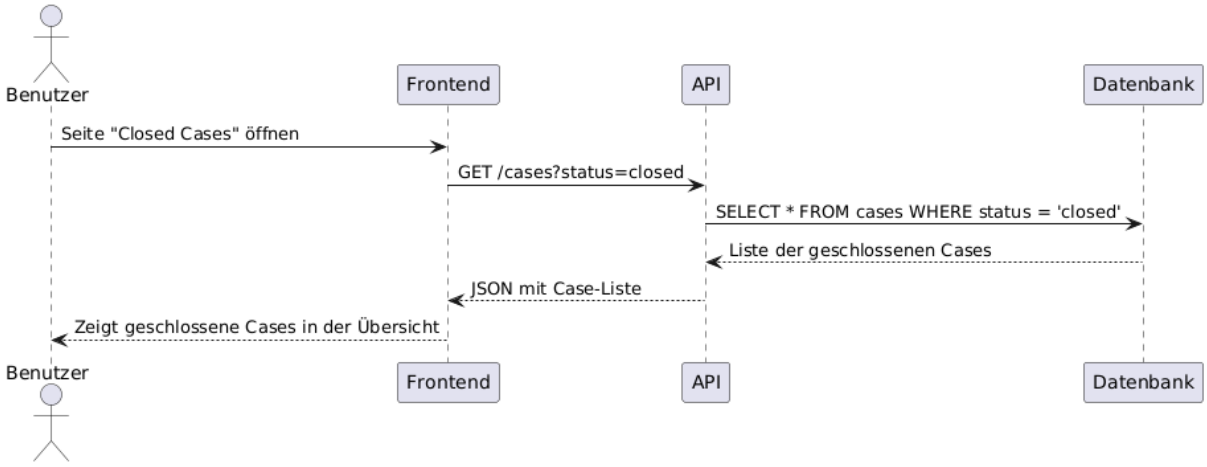


Abbildung 14 Sequenzdiagramm geschlossene Cases anzeigen

(PlantUML Web Server, n.d.)

Case Details anzeigen
Ein Benutzer möchte die Detailansicht eines bestimmten Cases einsehen, um genaue Informationen über den Vorfall zu erhalten, wie:

- Welche Incidents sind diesem Case zugehörig
- Wo im File kann er die sensitiven Daten finden

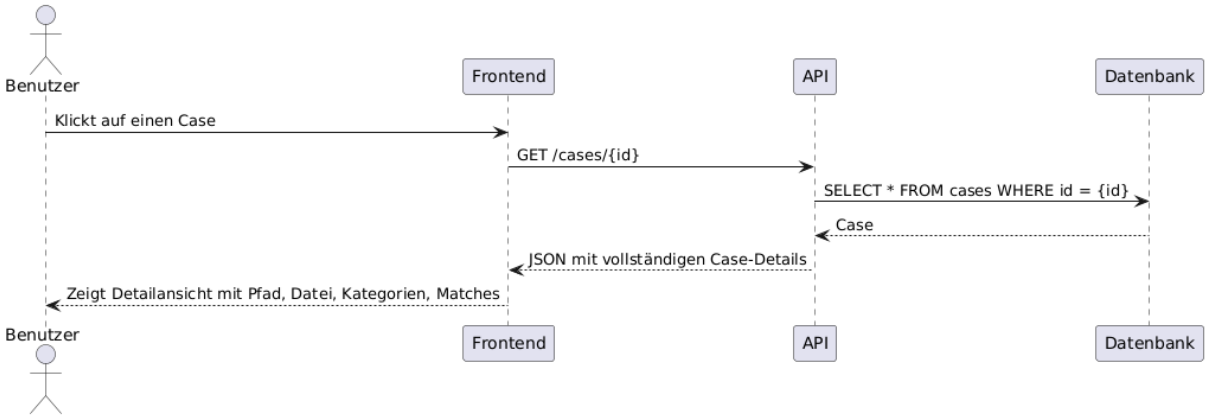


Abbildung 15 Sequenzdiagramm Case Details anzeigen

(PlantUML Web Server, n.d.)

Case als "remediated" markieren und schliessen
Ein Benutzer möchte ein Case als behoben markieren, nachdem er die Sensitiven Daten behoben hat. Dabei fügt er optional einen Kommentar hinzu.

Case als "False-Positive" markieren und schliessen
Ein Benutzer möchte ein Case als "False-Positive" markieren, wenn er feststellt, dass es sich bei den gefundenen Daten nicht um Sensitive Daten handelt und der Scanner diese fälschlicherweise gesichtet hat. Dabei fügt er optional einen Kommentar hinzu.

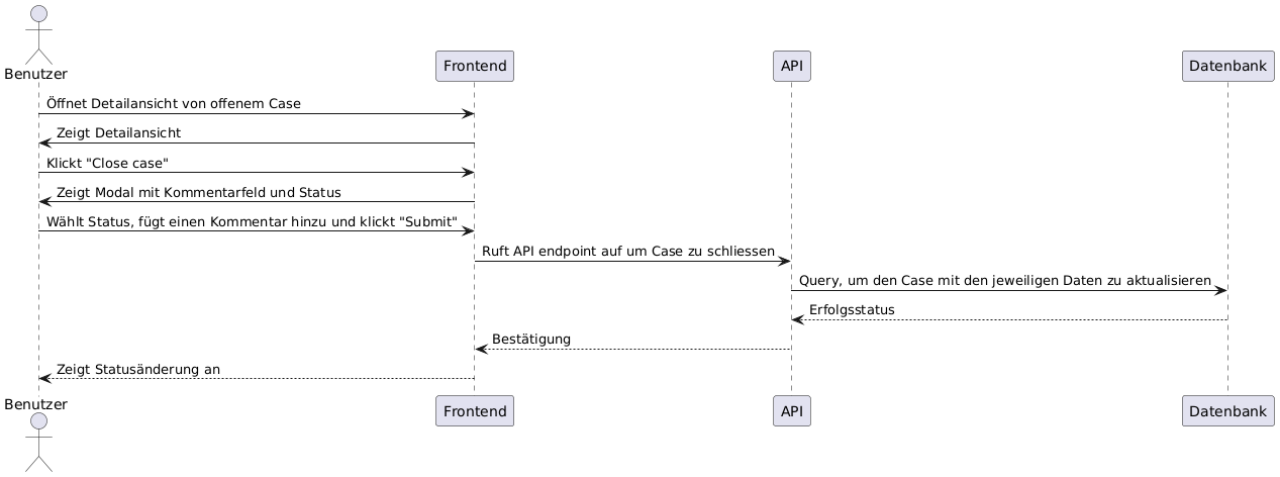


Abbildung 16 Sequenzdiagramm Case schliessen

(PlantUML Web Server, n.d.)

13.7.2 Komponenten

Die Applikation besteht aus folgenden Hauptkomponenten:

- Frontend: React GUI zur Anzeige und Verwaltung der Cases durch den Benutzer
- Backend: Python (FastAPI) REST-Service zur Bereitstellung und Verwaltung der Daten
- Datenbank: PostgreSQL mit definierter Tabellenstruktur
- Datenmigration: Python-Skript zur Übernahme der Daten aus JSON in die Datenbank

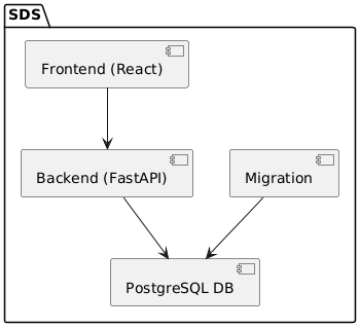
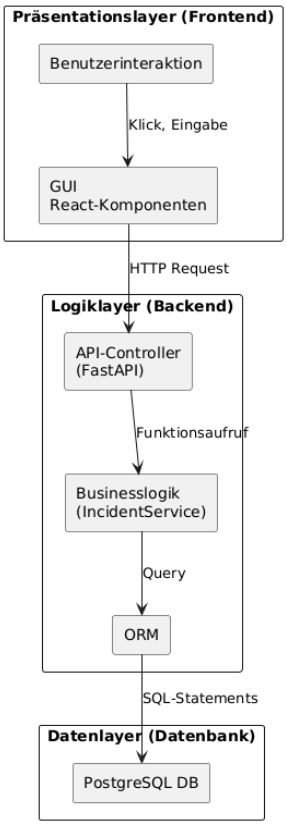


Abbildung 17 Komponenten
(PlantUML Web Server, n.d.)

13.7.3 Schichten



Die Architektur besteht aus einem 3-Schichtenmodell:

- Präsentationslayer (Frontend)
  - Darstellung der Daten in einem GUI, nach Mockup
  - Nutzerinteraktion
- Logiklayer (Backend)
  - API-Schnittstellen für Kommunikation mit Frontend
- Datenlayer (Datenbank)
  - Speicherung der Cases, Incidents und Matches

(PlantUML Web Server, n.d.)

Abbildung 18 Schichten

13.7.4 Schnittstellen

API-Schnittstellen

- Alle Cases mit bestimmtem Status abrufen.
- Einzelnen Case mit allen zugehörigen Details abrufen.
- Status und Kommentar eines Cases ändern.

Die API-Schnittstellen werden in der Phase "Entscheiden" detailliert analysiert und beschrieben. Basierend auf diesen Erkenntnissen werden entsprechende Entscheidungen getroffen.

In der Phase "Realisieren" werden die einzelnen Endpoints implementiert und zusätzlich im Detail dokumentiert und erläutert.

Migrations-Schnittstelle

- JSON-Datei aus dem SDS-Scan wird eingelesen.
- Relevante Daten werden extrahiert.
- Daten werden mittels eines geeigneten Tools in eine PostgreSQL Datenbank eingefügt.

In der Phase "Entscheiden" wird entschieden mit welchem Tool, die Migration durchgeführt wird.

In der Phase "Realisieren" wird dann die Migration mithilfe des bereits geplanten Migrationskonzept durchgeführt.

Backend-Datenbank-Schnittstelle

- Abfragen basierend auf API-Requests behandeln
- Zugriff erfolgt über ein ORM, welches SQL-Statements generiert und an die Datenbank sendet

13.7.5 Klassen

Mögliche Klassen für das Backend sind folgende:

CaseRouter

- Definiert REST-API-Endpoints für den Zugriff auf Cases
- Verarbeitet HTTP-Anfragen und leitet sie an die Businesslogik weiter

CaseService

- Zentrale Logik für Verarbeitung von Abfragen auf case

CaseModel, IncidentModel, MatchModel

- ORM-Modelle, welche die Tabellenstruktur der Datenbank abbilden.

CaseOverviewSchema, CaseDetailSchema, MatchSchema

- Pydantic-Models zur Validierung von Requests und Responses und als Schutz vor falscher Datenverarbeitung

13.7.6 Datenmodell

Die Erstellung des Datenbankmodells ist nicht Teil der IPA und wurde als Vorarbeit deklariert.

Datenbankstruktur

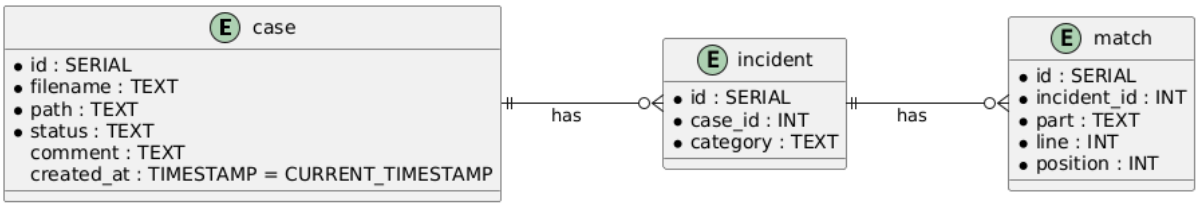


Abbildung 19 Datenbankstruktur
(PlantUML Web Server, n.d.)

Datenbankschema

```
CREATE TABLE "case" (
    id SERIAL PRIMARY KEY,
    filename TEXT NOT NULL,
    path TEXT NOT NULL,
    status TEXT not null,
    comment TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE incident (
    id SERIAL PRIMARY KEY,
    case_id INT NOT NULL,
    category TEXT NOT NULL,
    FOREIGN KEY (case_id) REFERENCES "case" (id) ON DELETE CASCADE
);

CREATE TABLE match (
    id SERIAL PRIMARY KEY,
    incident_id INT NOT NULL,
    part TEXT,
    line INT NOT NULL,
    position INT NOT NULL,
    FOREIGN KEY (incident_id) REFERENCES incident (id) ON DELETE CASCADE
);
```

14 Entscheiden

In der Phase "Entscheiden" werde ich alle offenen Punkte bezüglich des Projektes klären und mich für Varianten entscheiden.

Wo die Umsetzung noch unklar ist, werde ich die verschiedenen Varianten mithilfe einer Entscheidungsmatrix vergleichen und durch das Resultat festlegen, für welche Variante ich mich entscheide. Diese Variante wird dann in der Phase "Realisieren" umgesetzt.

14.1 Entscheidungsmatrix

Eine Entscheidungsmatrix ist ein hilfreiches Werkzeug, um mehrere vergleichbare Varianten objektiv miteinander zu vergleichen und den Entscheidungsprozess strukturiert und transparent darzustellen.

Vorgehen:

1. Optionen definieren:
   Zuerst werden zwei oder mehr vergleichbare Varianten identifiziert, zwischen denen eine Entscheidung getroffen werden soll.
2. Kriterien festlegen:
   Anschliessend werden relevante Kriterien bestimmt, mit denen die Varianten dann bewertet werden.
3. Wichtigkeit einstufen:
   Jedes Kriterium wird auf einer Skala von 1 bis 5 nach Wichtigkeit gewichtet
4. Varianten bewerten:
   Jede Variante wird für jedes Kriterium auf einer Skala von 1 bis 10 bewertet.
5. Gesamtbewertung berechnen:
   Die Bewertung einer Variante pro Kriterium wird mit der jeweiligen Gewichtung multipliziert. Anschliessend werden alle gewichteten Werte pro Variante addiert.

(Martins, 2025)

14.2 Entscheidungsmatrix Datenmigration

14.2.1 Varianten

ETL-Tool

Extract, Transform, Load (ETL) ist ein Prozess, um Daten aus verschiedenen Quellen in eine grosses, zentrales Repository zusammenzufügen.

Der ETL-Prozess läuft in drei Schritten ab:

1. Extrahieren der relevanten Daten aus der Quelldatenbank
2. Umwandeln der Daten, so dass sie besser für Analytik geeignet sind
3. Laden der Daten in die Zieldatenbank

(AWS, 2025)

Python-Skript (psycopg2)

Ein Python-Skript liest das SDS JSON-File ein, verarbeitet die Daten und schreibt sie mithilfe von psycopg2 in die PostgreSQL-Datenbank.

Der Prozess:

1. Struktur der Daten analysieren und einordnen
2. Skript schreiben, welches das SDS JSON-File entsprechend parst
3. Mit psycopg2 eine Verbindung zur Datenbank aufbauen und SQL-Statements ausführen

14.2.2 Kriterien

Kriterium 1: Vorkenntnisse
Bewertet, wie gut ich mich mit der jeweiligen Variante bereits auskenne. Je höher der Wert, desto besser sind meine bestehenden Kenntnisse.

Kriterium 2: Implementationszeit
Schätzt den Zeitaufwand für die Umsetzung der Variante. Ein hoher Wert bedeutet, dass die Lösung in kurzer Zeit realisierbar ist.

Kriterium 3: Flexibilität
Bewertet, wie gut sich die Lösung an neue Anforderungen, unterschiedliche Datenstrukturen oder künftige Änderungen anpassen lässt.

Kriterium 4: Zuverlässigkeit
Bewertet, wie stabil und fehlerfrei die Lösung im Einsatz voraussichtlich funktionieren wird.

Kriterium 5: Fehlerbehandlung
Bewertet, wie gut sich Fehler identifizieren und beheben lassen.

14.2.3 Entscheidungsmatrix

	Kriterium 1 *2	Kriterium 2 *3	Kriterium 3 *1	Kriterium 4 *3	Kriterium 5 *3	End-resultat
ETL-Tool	1*2=2	3*3=9	8*1=8	10*3=30	7*3=21	**70**
Python-Skript	7*2=14	8*3=24	3*1=3	7*3=21	7*3=21	**83**

Tabelle 36 Entscheidungsmatrix Datenmigration

14.3 API-Schnittstellen

14.3.1 Alle Cases abrufen

Attribut	Beschreibung
Methode	GET
Pfad	/cases
Query-Parameter	`status` (erforderlich): open oder closed `offset` (optional, Standard: 0): Startindex der Daten `limit` (optional, Standard: 10): Anzahl der Einträge
Request-Body	keiner
Response	Liste der Cases im angegebenen Status, paginiert

Tabelle 37 Endpoint alle Cases

14.3.2 Case Details abrufen

Attribut	Beschreibung
Methode	GET
Pfad	/cases/{id}
Pfad-Parameter	id: int
Request-Body	keiner
Response	JSON mit allen Details zum Case (inkl. Incidents, Matches)

Tabelle 38 Endpoint Case Detail

14.3.3 Case schliessen

Dieser Endpoint könnte mit den Methoden POST, PUT oder PATCH umgesetzt werden. Welche Methode ich wähle, werde ich im nächsten Abschnitt 14.4 mit einer Entscheidungsmatrix definieren.

14.4 Entscheidungsmatrix Case schliessen Endpoint

14.4.1 Varianten

- **POST-Methode**
- **PUT-Methode**
- **PATCH-Methode**

14.4.2 Kriterien

Kriterium 1: REST-Konform
Bewertet, ob die gewählte Methode sinnvoll zu dem passt, was damit gemacht werden soll und den REST-Standards entspricht.

Kriterium 2: Flexibilität
Bewertet, wie einfach sich nur bestimmte Felder (z. B. status, comment) ändern lassen, ohne die gesamte Ressource übermitteln zu müssen.

Kriterium 3: Klarheit des Endpoints
Bewertet, wie verständlich und eindeutig der Zweck des Endpoints ist.

Kriterium 4: Umsetzung
Bewertet, wie gut die Methode zur bereits geplanten Architektur passt.

14.4.3 Entscheidungsmatrix

Methode	Kriterium 1 *2	Kriterium 2 *2	Kriterium 3 *1	Kriterium 4 *3	Gesamtpunktzahl
POST	4*2=8	5*2 = 10	9*1=9	10*3=30	57
PUT	6*2=12	3*2=6	6*1=6	6*3=18	42
PATCH	10*2=20	10*2 = 20	8*1=8	8*3=24	72

Tabelle 39 Entscheidungsmatrix Case schliessen

14.4.4 Endpoint Case schliessen

Attribut	Beschreibung
Methode	PATCH
Pfad	/cases/{id}
Pfad-Parameter	id: int
Request-Body	JSON: `{"comment": "...", "status": "remediated"}`
Response	Bestätigung oder aktualisiertes Case-Objekt

Tabelle 40 Endpoint Case schliessen

15 Realisieren

15.1 Auswahl

Die eingesetzten Tools und Frameworks wurden entsprechend den firmeninternen Vorgaben sowie gängigen Best Practices ausgewählt und eingerichtet:

- Die verwendeten Frameworks FastAPI und React wurden im Vorfeld als technologische Basis festgelegt.
- Als Entwicklungsumgebung nutze ich Visual Studio Code, da ich mit der Bedienung bestens vertraut bin und ich das Tool nach meinen Bedürfnissen erweitern kann, was mir einen produktiveren Workflow ermöglicht.
- Die Versionskontrolle erfolgt über git bzw. Gitlab, da dies der firmenweite Standard ist und wir eine eigene Gitlab Instanz besitzen.
- Die Wahl des Datenbank-Clients ist nicht vorgegeben, solange die Software unter einer freien Lizenz steht. Ich verwende hier DBeaver, da es eine intuitive Benutzeroberfläche bietet und unter der Apache 2.0 Lizenz frei genutzt werden darf.
- Zur Effizienzsteigerung in der Entwicklung habe ich in VS Code zusätzliche Extensions installiert:
    - **Pylance** für fortschrittliches Python-Linting und Typprüfung
    - **Turbo Console Log** für schnelleres Debugging in JavaScript-/React-Projekten
    - **Autopep8**, die aktiv den Code auf Einhaltung der PEP 8 Best Practices überprüft

15.2 Setup

Dieser Abschnitt ist eine Schritt-für-Schritt-Anleitung, für das Setup des Projektes auf einem Windows 11 Gerät. Sie enthält detaillierte Anweisungen für die Installation aller notwendigen Tools und Dienste.

15.2.1 Datenbank

1. Postgres Installer von der offiziellen Webseite herunterladen und starten: https://www.postgresql.org/download/
2. Den Anweisungen des Installers folgen und Passwort für den Default User setzen.
3. Testen, ob PostgreSQL läuft, indem man im Terminal folgendes eingibt: `psql -U postgres`
4. Datenbank in der Kommandozeile erstellen: `createdb -U postgres ipa`
5. Verifizieren, dass die Datenbankerstellt wurde in der psql Shell: `\list`
6. Verbindung zur Datenbank in DBeaver herstellen.
7. SQL-Skript ausführen, welches das Datenbankschema enthaltet (**Datenmodell**).

15.2.2 Entwicklungsumgebung

1. VS Code Installer herunterladen von der offiziellen Webseite: https://code.visualstudio.com/
2. Installer ausführen und den Anweisungen folgen
3. Visual Studio Code starten
4. Erweiterungen in VSCode installieren, die den Entwickler beim Entwicklungsprozess unterstützen:
   a. Python (Microsoft)
   b. Pylance
   c. Python Debugger
   d. autopep8
   e. ESLint
   f. Prettier
   g. Turbo Console Log

   Bei den Erweiterungen schaue ich darauf, dass sie von verifizierten Herausgebern entwickelt worden sind, viele Downloads haben und ausschliesslich positiv bewertet sind. So muss ich keine sicherheitsrelevanten Bedenken haben beim Installieren und Verwenden dieser Erweiterungen.

15.2.3 Python

1. Aktuellste Version von Python herunterladen von der offiziellen Webseite: https://www.python.org/downloads/
2. Installer starten und Anweisungen folgen
3. "Add Python to PATH" aktivieren

15.2.4 Node.js

1. LTS-Version von https://nodejs.org/en herunterladen und installieren
2. Anweisungen des Installers folgen
3. Im Terminal testen mit: `node -version`

15.2.5 Git

1. Git von der offiziellen Webseite herunterladen und installieren: https://git-scm.com/downloads
2. Anweisungen des Installers folgen
3. Git testen mit: `git -version`
4. Git konfigurieren:

```
git config --local user.name "Benjamin Scheibler"
git config --local user.email "benjamin.scheibler@swisscom.com"
```

15.2.6 Python Virtual Environment

Erklärung

Eine virtuelle Umgebung oder Virtualenv ermöglicht es, mehrere parallele Instanzen des Python-Interpreters zu starten. Jede dieser Instanzen hat ihren eigenen Satz an Paketen und ihre eigenen Konfigurationen. Die virtuelle Umgebung enthält auch eine Kopie des Python-Interpreters.
Die in jeder Umgebung installierten Pakete sind nur in dieser virtuellen Umgebung zu sehen und nirgendwo sonst.

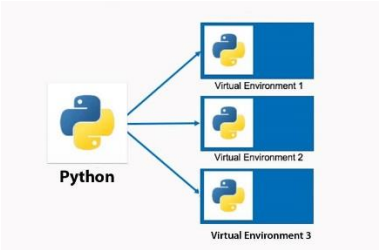


Abbildung 20 Virtuelle Umgebung
(Daniel, 2023)

Installation

1. In Projektverzeichnis navigieren
2. Virtualenv erstellen: `python -m venv venv`
3. Virtualenv aktivieren: `venv\Scripts\activate`
4. FastAPI und uvicorn installieren: `pip install fastapi uvicorn`

15.2.7 Umgebungsvariablen

Umgebungsvariablen werden verwendet, um die Konfigurationsvariablen, wie zum Beispiel das Datenbankpasswort lokal zu behalten und im ganzen Projekt zu verwenden, ohne dass diese öffentlich zugänglich sind. Die Umgebungsvariablen in diesem Projekt sehen folgendermassen aus:

```
$env:DB_USERNAME="postgres"
$env:DB_NAME="ipa"
$env:DB_USER="postgres"
$env:DB_PASSWORD="root"
$env:DB_HOST="localhost"
$env:DB_PORT="5432"
```

Mit `.\filename` kann man die Variablen im Projekt setzen und mit dem os Package holen, z.B. mit `os.getenv('DB_USER')`

15.3 Datenmigration

15.3.1 Ablauf

1. Psycopg2 installieren: `pip install psycopg2-binary`
2. Umgebungsvariablen setzen: `./envvariables`
3. Migrationsskript ausführen
4. In der Datenbank kontrollieren, ob die Daten korrekt und vollständig importiert worden sind

15.3.2 Kontrolle in der Datenbank

Nach der Migration müssen die Daten in der Datenbank vollständig und korrekt enthalten sein.

Die Kontrolle der migrierten Daten wird in der Phase "Kontrolle" genauer untersucht und überprüft, indem Funktions -und Performancetests durchgeführt werden.

15.3.3 Migrationsskript

Das Skript importiert als erstes alle nötigen Libraries, stellt eine Verbindung zur Datenbank her und öffnet dann den JSON-Report.

```
import psycopg2
import json
import os

# Database connection parameters
conn = psycopg2.connect(
    dbname=os.getenv('DB_NAME'),
    user=os.getenv('DB_USER'),
    password=os.getenv('DB_PASSWORD'),
    host=os.getenv('DB_HOST'),
    port=os.getenv('DB_PORT')
)
cur = conn.cursor()

# Open the SDS Report JSON file and load the data
with open('import/AA_Result-DB-FileScanX.json') as f:
    data = json.load(f)
```

Danach wird durch jeden Eintrag des "matches" Array im Report-JSON iteriert.

```
for entry in data["Matches"]:
    case_id = entry["S"]
    filename = entry["FileName"]
    path = entry["Path"]
    category = entry["Category"]
    part = entry["Part"].strip()
    line = int(entry["Line"])
    position = int(entry["Position"])
```

Wenn noch kein Case mit diesem File und Path existiert, wird ein neuer Case erstellt.

```
# Insert into the case table (if not already exists)
    cur.execute('SELECT id FROM "case" WHERE path = %s and filename = %s',
(path, filename))
    case_exists = cur.fetchone()
    if not case_exists:
        cur.execute(
            'INSERT INTO "case" (id, path, filename, status) VALUES (%s,
%s, %s, %s)',
            (case_id, path, filename, "open")
        )
```

Dasselbe passiert mit Incident und Match.

```
    # Insert incidents
    cur.execute(
        'SELECT id FROM incident WHERE case_id = %s AND category = %s',
        (case_id, category)
    )
    incident_id = cur.fetchone()
    if not incident_id:
        cur.execute(
            'INSERT INTO incident (case_id, category) VALUES (%s, %s)',
            (case_id, category)
        )
        cur.execute(
            'SELECT id FROM incident WHERE case_id = %s AND category =
%s',
            (case_id, category)
        )
        incident_id = cur.fetchone()
        print(f"Inserted incident for case {case_id} with category
{category}")

    # Insert matches
    cur.execute('SELECT id FROM "match" WHERE incident_id = %s AND part =
%s AND line = %s AND "position" = %s',
            (incident_id, part, line, position))
    match_id = cur.fetchone()

    if not match_id:
        cur.execute(
            'INSERT INTO "match" (incident_id, part, line, "position")
VALUES (%s, %s, %s, %s) RETURNING id',
            (incident_id, part, line, position)
        )
        match_id = cur.fetchone()
        print(f"Inserted match with id {match_id} for incident
{incident_id}")
```

Anschliessend werden die Queries ausgeführt und eine Bestätigung wird in der Konsole ausgegeben, wenn alles erfolgreich durchlaufen ist.

```
conn.commit()
cur.close()
conn.close()

print("Data insertion complete!")
```

15.4 Backend

15.4.1 Setup

Das zuvor erstellte virtaulenv aktivieren und FastAPI, sowie uvicorn (Server) darin installieren:

`.venv\Scripts\activate`

`pip install fastapi uvicorn`

Minimale FastAPI Applikation mit Test Endpoint erstellen:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def hello_world():
    return {"Hello": "World"}
```

Server starten mit: `uvicorn app.main:app`

Wenn auf http://127.0.0.1:8000/ eine erfolgreiche Response vom Server zu sehen ist, ist das Backend korrekt aufgesetzt.

Grundlegende Ordnerstruktur erstellt für bessere Übersicht und Trennung erstellen:

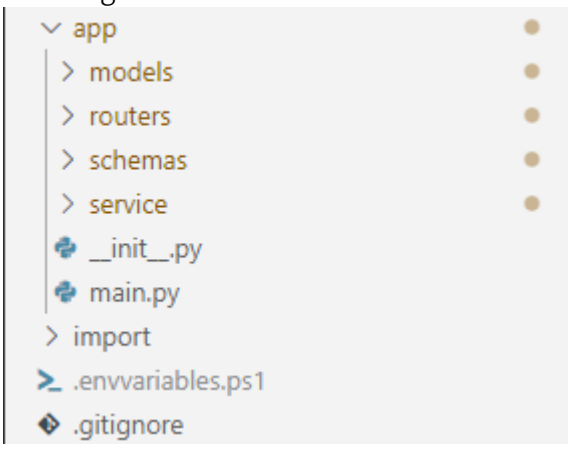


Abbildung 21 Ordnerstruktur Backend

/app/routers: In diesem Verzeichnis befinden sich die Dateien mit den API-Endpoints.

/app/services: Hier befindet sich die Business-Logik. Hier findet der Zugriff über ein ORM auf die Datenbank statt.

/app/schemas: Dieses Verzeichnis enthält die pydantic Models, welche beispielsweise die JSON-Struktur der Case Response vorgibt.

/app/models: Verzeichnis mit SQLAlchemy Datenbankmodellen. Sie simulieren die Struktur der Datenbank mit Python-Klassen. Dies erlaubt es mit der Datenbank, wie mit normalen Python-Objekten zu arbeiten. So muss man nicht direkte SQL-Queries direkt an die Datenbank senden.

15.4.2 Konfiguration

Um den Entwicklungsprozess zu unterstützen und die Einhaltung der Code-Richtlinien sicherzustellen, habe ich die VS Code Erweiterung autopep8 installiert.
Die VS Code Einstellungen wurden so konfiguriert, dass der Code bei jedem Speichern automatisch formatiert wird. Dies stellt automatisch sicher, dass die definierten Code-Richtlinien eingehalten werden, und fördert die Effizienz:

```
    "[python]": {
        "editor.defaultFormatter": "ms-python.autopep8",
        "editor.formatOnSave": true
      }
```

Durch die installierte Python Debugger Erweiterung kann ich regelmässig und strukturiert meinen Code debuggen, wenn es zu Problemen kommen sollte.

15.4.3 SwaggerUI Dokumentation

FastAPI stellt automatisch eine SwaggerUI Dokumentation der implementierten Endpoints zur Verfügung, die unter http://localhost:8000/docs erreichbar ist, sobald der Server läuft.

SwaggerUI ist eine Benutzeroberfläche, welche die API-Endpoints nach OpenAPI Standards strukturiert darstellt. Auf dieser Benutzeroberfläche können die Endpoints direkt im Browser getestet werden.

Durch diese Benutzeroberfläche kann ich meine Endpoints strukturiert, regelmässig und effizient testen und gleichzeitig meinen Code debuggen.

15.4.4 Datenbankverbindung

Zur Kommunikation mit der PostgreSQL Datenbank wird das Package `sqlmodel` verwendet, welches auf SQLAlchemy aufbaut und pydantic models unterstützt.
Die Datenbankverbindung wird mit den zuvor definierten Umgebungsvariablen in Kapitel definiert:

```
db_url =
f"postgresql://{os.getenv('DB_USER')}:{os.getenv('DB_PASSWORD')}@{os.geten
v('DB_HOST')}:{os.getenv('DB_PORT')}/{os.getenv('DB_NAME')}"
```

Mit dieser URL wird die "sqlmodel engine" erzeugt, welche die zentrale Schnittstelle zur Datenbank ist:

```
engine = create_engine(db_url)
```

Um mit der Datenbank zu interagieren wird eine Session benötigt.

```
def get_session():
    with Session(engine) as session:
        yield session
```

In den folgenden Endpoints wird diese Session mithilfe von Dependency Injection verwendet.

15.4.5 Alle Cases abrufen

Attribut	Beschreibung
Methode	GET
Pfad	/cases
Query-Parameter	`status` (erforderlich): open oder closed `offset` (optional, Standard: 0): Startindex der Daten `limit` (optional, Standard: 10): Anzahl der Einträge
Request-Body	keiner
Response	Liste der Cases im angegebenen Status, paginiert

Tabelle 41 Endpoint alle Cases

Anforderungen abzudecken von Kapitel 12.1.1: **F-02.1 F-02.3, F-08**

Implementierung im Backend:

```
# /routers/case.py

@case_router.get("/")
def get_cases(
    session: Annotated[Session, Depends(get_session)],
    status: CaseStatus = CaseStatus.open,
    offset: int = 0,
    limit: int = 10
) -> list[CaseOut]:
    try:
        return fetch_cases(session, status, offset, limit)
    except Exception as e:
        raise HTTPException(
            status_code=400, detail=f"Error fetching cases: {e}",)
```

```
# /service/case.py

def fetch_cases(session, status: str = "open", offset: int = 0, limit: int
= 10):
    try:
        if status == "closed":
            cases = session.query(Case).filter(
                Case.status.in_(["false-positive",
"remediated"])).offset(offset).limit(limit).all()
        else:
            cases = session.query(Case).filter(
                Case.status == "open").offset(offset).limit(limit).all()
        return cases
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

Mit dem Hinzufügen der Query Parameters `offset` und `limit`, wurde eine funktionierende Pagination der Cases hinzugefügt und somit der Backend Teil von Anforderung F-08 abgedeckt. Um diese Anforderung vollständig abzuschliessen, muss die Pagination noch im Frontend umgesetzt werden.

SwaggerUI Dokumentation, welche von FastAPI automatisch erstellt wird:

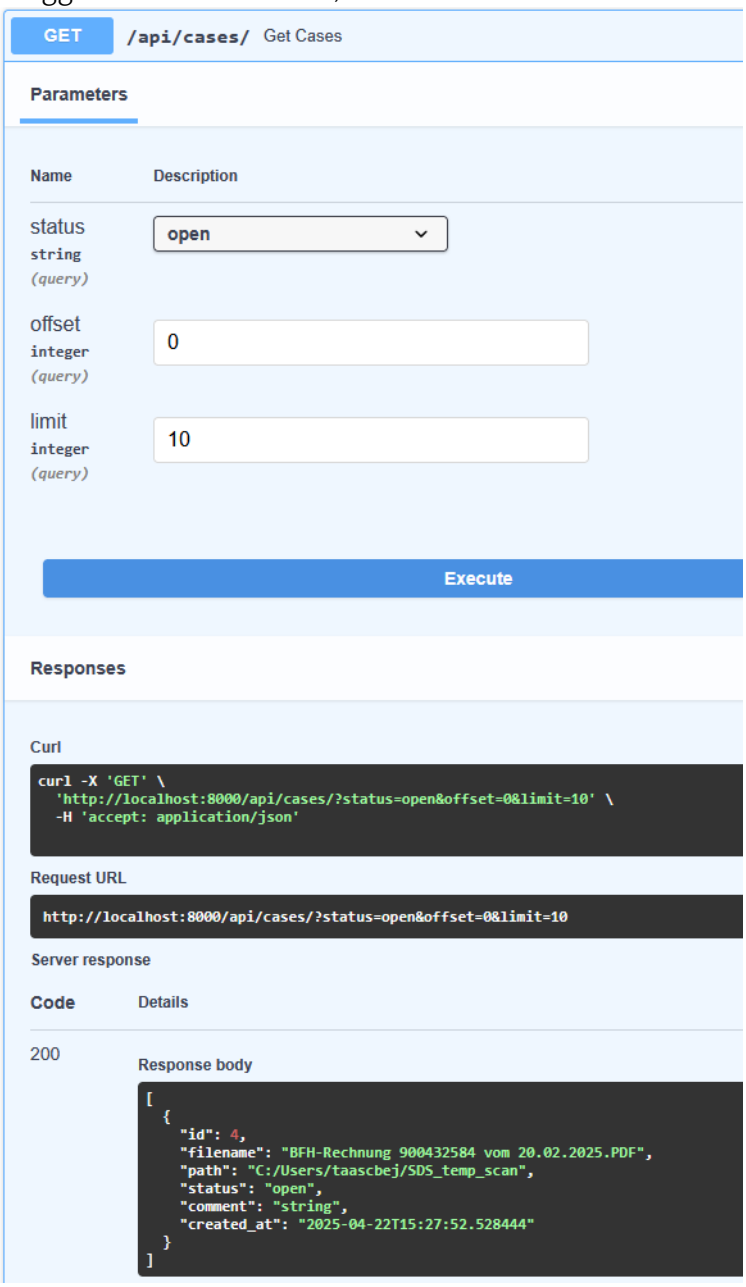


Abbildung 22 SwaggerUI alle Cases

15.4.6 Case Details abrufen

Attribut	Beschreibung
Methode	GET
Pfad	/cases/{id}
Pfad-Parameter	id: int
Request-Body	keiner
Response	JSON mit allen Details zum Case (inkl. Incidents, Matches)

Tabelle 42 Endpoint Case Detail

Anforderungen abzudecken von Kapitel 12.1.1: **F-02.2, F-04**

Implementierung im Backend:

```
# /routers/case.py

@case_router.get("/{case_id}/")
async def get_case(case_id: int, session: Annotated[Session,
Depends(get_session)]) -> CaseDetail:
    try:
        return fetch_case_by_id(case_id, session)
    except HTTPException as e:
        raise e
    except Exception:
        raise HTTPException(
            status_code=400, detail=f"Error fetching case")
```

```
# /service/case.py

def fetch_case_by_id(case_id: int, session):
    case = session.query(Case).filter(Case.id == case_id).first()
    if case is None:
        raise HTTPException(
            status_code=404, detail=f"Case with id {case_id} not found")
    return case
```

SwaggerUI Dokumentation:

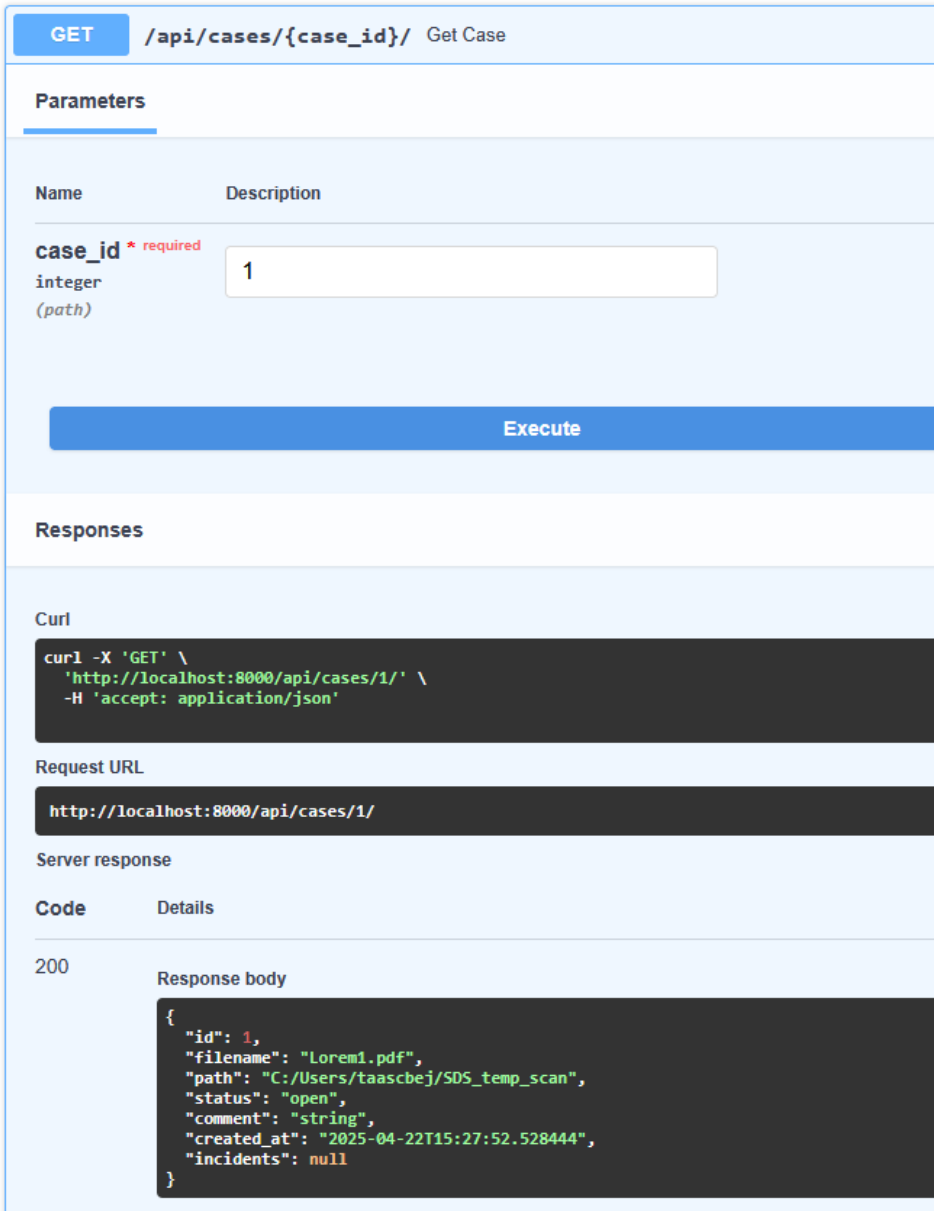


Abbildung 23 SwaggerUI Case Detail

15.4.7 Case schliessen

Attribut	Beschreibung
Methode	PATCH
Pfad	/cases/{id}
Pfad-Parameter	id: int
Request-Body	JSON: `{"comment": "...", "status": "remediated"}`
Response	Bestätigung oder aktualisiertes Case-Objekt

Tabelle 43 Endpoint Case schliessen

Anforderungen abzudecken von Kapitel 12.1.1: **F-02.4, F-06, F-07**

Backend Implementierung:

```
# /routers/case.py

@case_router.patch("/{case_id}/")
async def close_case(case_id: int, patch_case: PatchCase, session:
Annotated[Session, Depends(get_session)]) -> CaseDetail:
    try:
        return close_case_by_id(case_id, patch_case, session)
    except HTTPException as e:
        raise e
    except Exception:
        raise HTTPException(
            status_code=400, detail=f"Error closing case")
```

```
# /service/case.py

def close_case_by_id(case_id: int, patch_case, session):
    case = session.query(Case).filter(Case.id == case_id).first()
    if case is None:
        raise HTTPException(
            status_code=404, detail=f"Case with id {case_id} not found")
    if case.status != "open":
        raise HTTPException(
            status_code=400, detail=f"Case with id {case_id} is already
closed")
    case.status = patch_case.status
    case.comment = patch_case.comment
    session.commit()
    return case
```

SwaggerUI Dokumentation:

PATCH /api/cases/{case_id}/ Close Case

Parameters

Name	Description
case_id * required integer (path)	3

Request body required

```
{
  "status": "false-positive",
  "comment": "comment for screenshot"
}
```

Execute

Responses

Curl

```
curl -X 'PATCH' \
  'http://localhost:8000/api/cases/3/' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "status": "false-positive",
  "comment": "comment for screenshot"
}'
```

Request URL

```
http://localhost:8000/api/cases/3/
```

Server response

Code	Details
200	Response body

```
{
  "id": 3,
  "filename": "Lorem1.docx",
  "path": "C:/Users/taascbej/SDS_temp_scan",
  "status": "false-positive",
  "comment": "comment for screenshot",
  "created_at": "2025-04-22T15:27:52.528444",
  "incidents": null
}
```

Abbildung 24 SwaggerUI Case schliessen

15.4.8 Pagination

Um die Pagination im Frontend funktionsfähig zu implementieren, benötigte ich die Gesamtzahl der Cases mit dem jeweiligen Status. Aus diesem Grund habe ich noch einen weiteren Backend API-Endpoint implementiert, um die Anzahl Cases zu bekommen.

Anforderungen abzudecken von Kapitel 12.1.1: **F-08**

Implementation Backend

```
# /routers/case.py

@case_router.get("/caseCount")
def get_case_count(
    session: Annotated[Session, Depends(get_session)],
    status: CaseStatus = CaseStatus.open
):
    try:
        case_count = fetch_case_count(session, status)
        return {"case_count": case_count}
    except Exception as e:
        raise HTTPException(
            status_code=400, detail=f"Error fetching case count: {e}")
```

```
# /service/case.py

def fetch_case_count(session, status: CaseStatus = "open"):
    try:
        if status == "closed":
            case_count = session.query(Case).filter(
                Case.status.in_(["false-positive", "remediated"])).count()
        else:
            case_count = session.query(Case).filter(
                Case.status == "open").count()
        return case_count
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

SwaggerUI Dokumentation

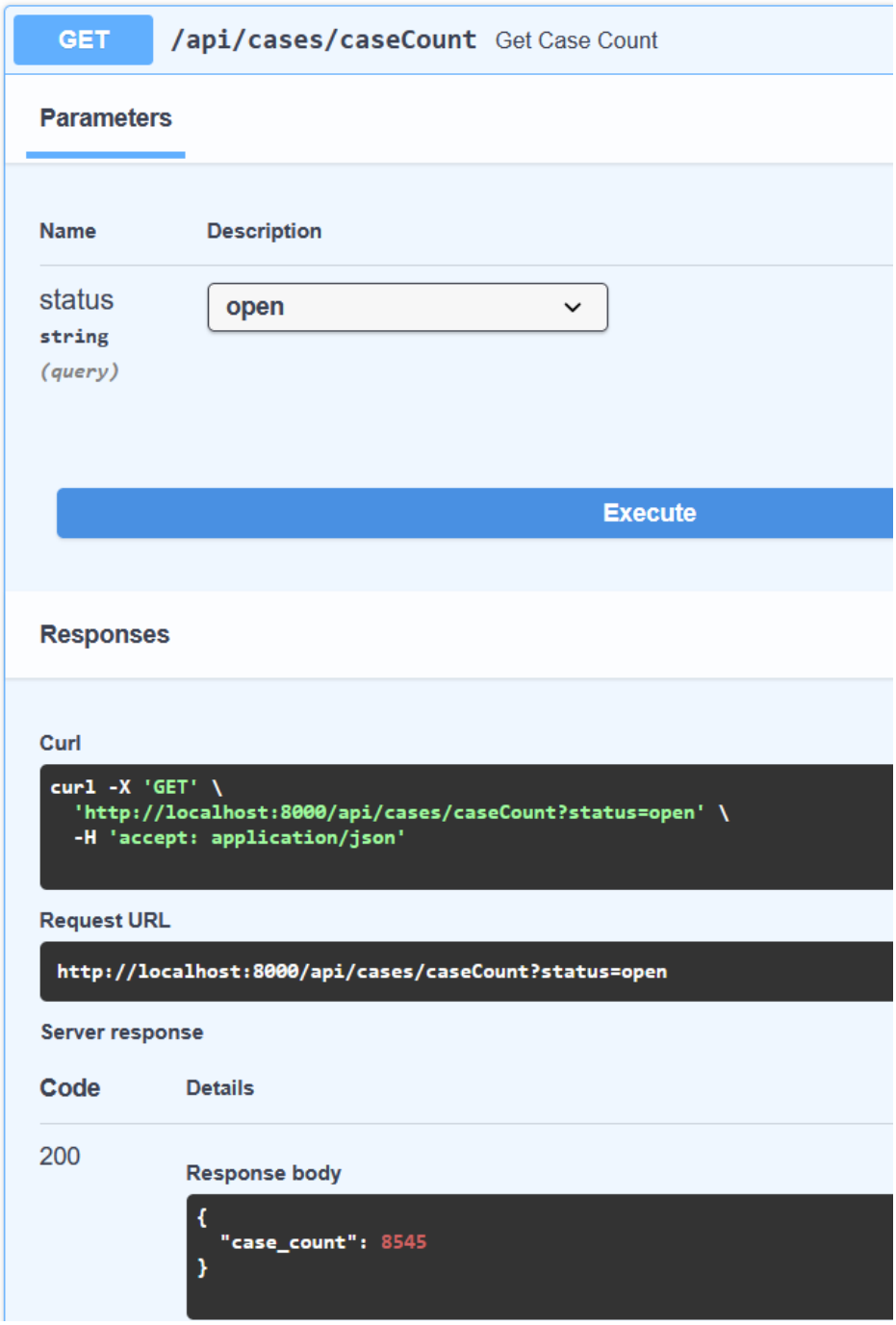


Abbildung 25 SwaggerUI Anzahl Cases

15.5 Frontend

15.5.1 Setup

Es besteht eine React Grundapplikation, welche die SDX-Komponenten beinhaltet. Ich habe dieses Repository geklont und mein erstelltes Git Repository mit diesem bestehenden Projekt konfiguriert:

```
git init --initial-branch=main
git remote add origin
ssh://git@code.swisscom.com:2222/benjamin.scheibler/ipa-frontend.git
git add .
git commit -m "Initial commit"
git push --set-upstream origin main
```

Mit `npm install` habe ich dann alle benötigten Packages für dieses Projekt installiert.

Mit `npm start` konnte ich den Development Server starten und testen, ob die Applikation unter http://localhost:3000 erreichbar ist. Wenn das der Fall ist, ist das Frontend korrekt und vollständig aufgesetzt.

15.5.2 Konfiguration

Um mich als Entwickler im Entwicklungsprozess zu unterstützen, habe ich die in Kapitel 0 installiert beschriebenen VS Code Erweiterungen installiert.
Zusätzlich habe ich die Einstellungen so angepasst, dass mein Code automatisch formatiert wird, sobald ich ihn mit CTRL+S speichere.:

```
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  }
```

15.5.3 Offene Cases Übersicht

Anforderungen abzudecken: NF-01, F-03, F-05, F-08

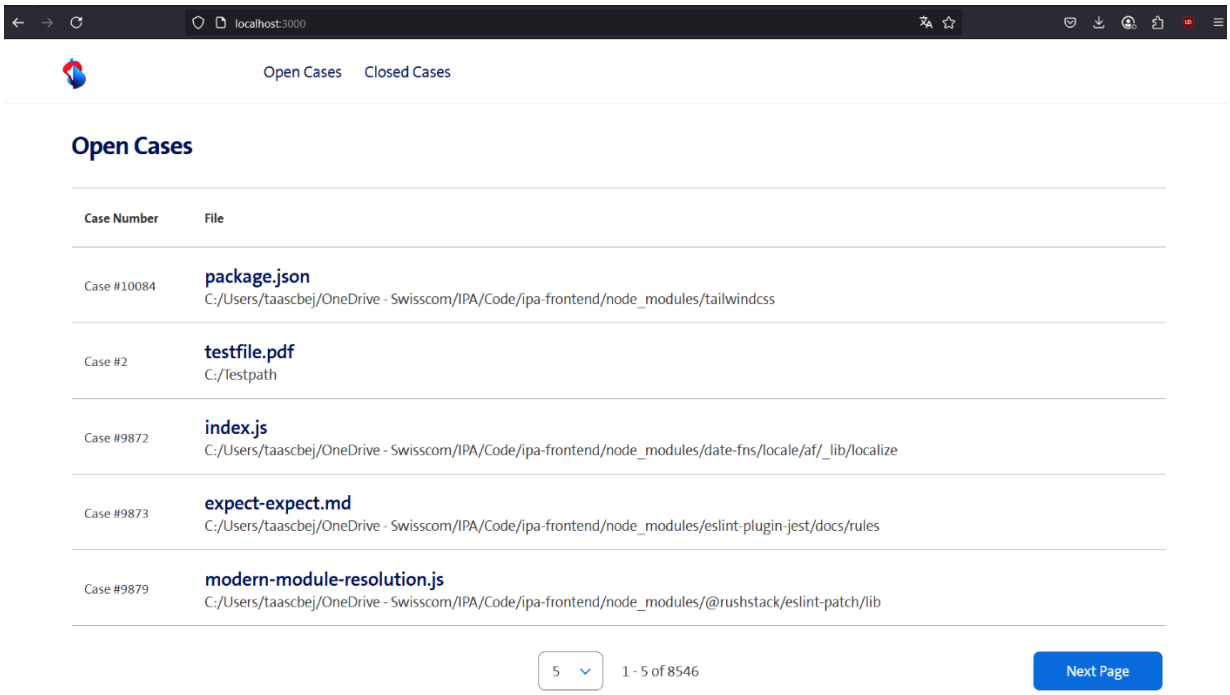


Abbildung 26 Frontend offene Cases

15.5.4 Geschlossene Cases Übersicht

Anforderungen abzudecken: NF-01, F-03, F-05, F-08

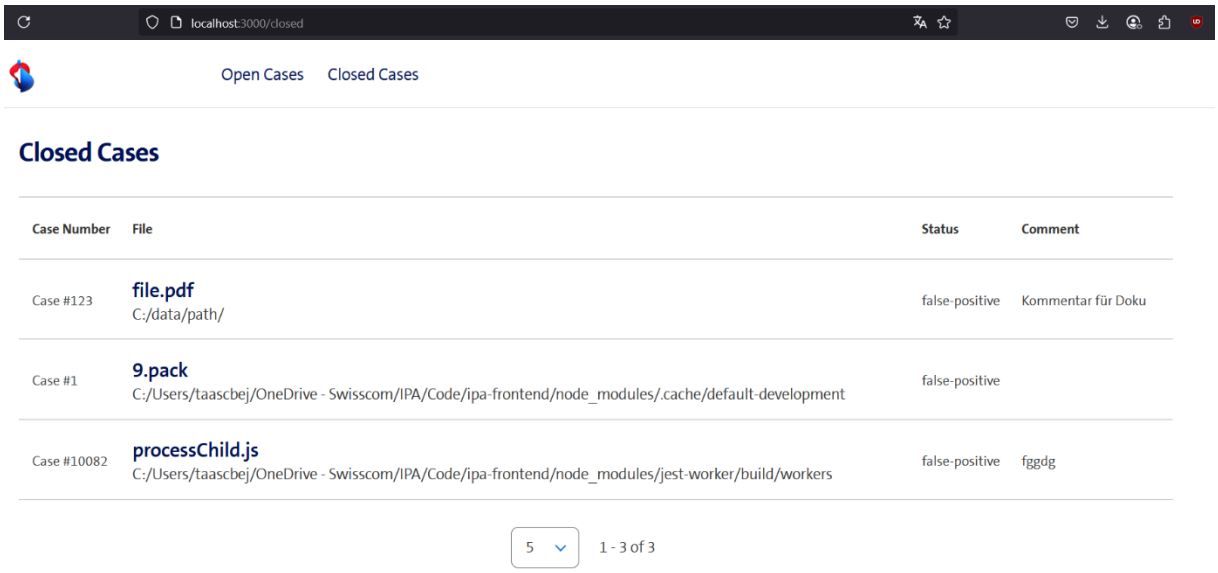


Abbildung 27 Frontend geschlossene Cases

15.5.5 Case Detailansicht

Anforderungen abzudecken: NF-01, F-04

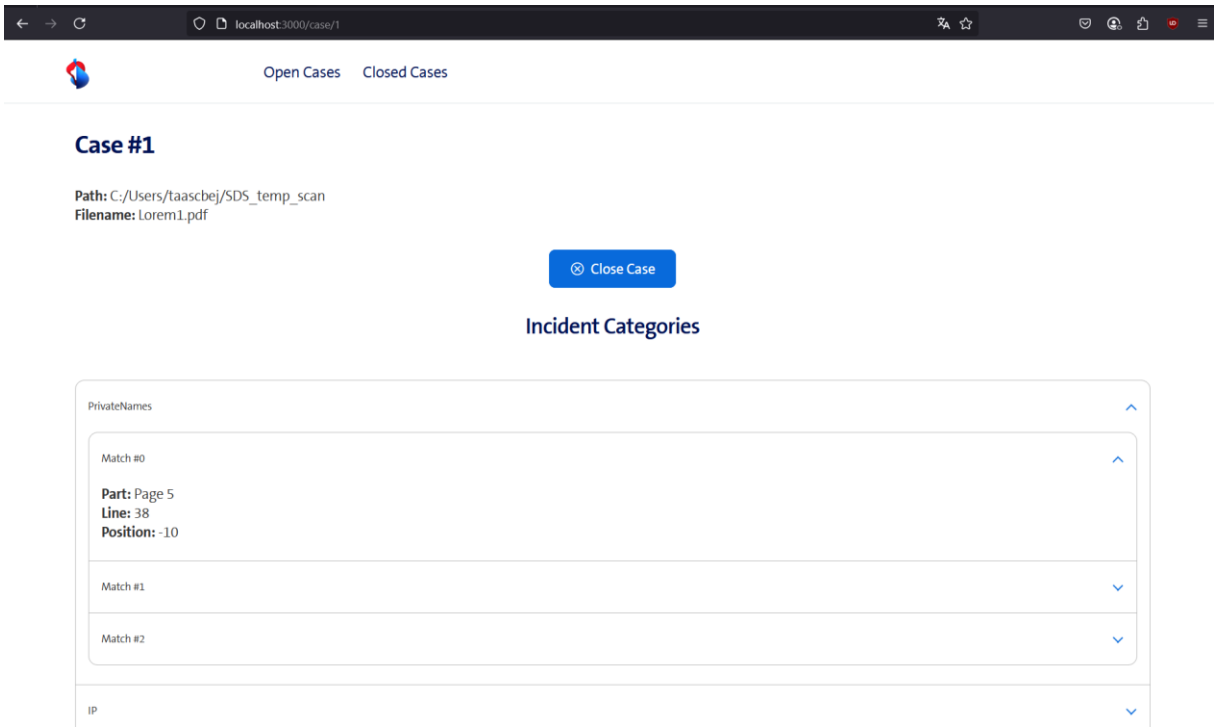


Abbildung 28 Frontend Case Detail

15.5.6 Case schliessen Modal

Anforderungen abzudecken: NF-01, F-06, F-07

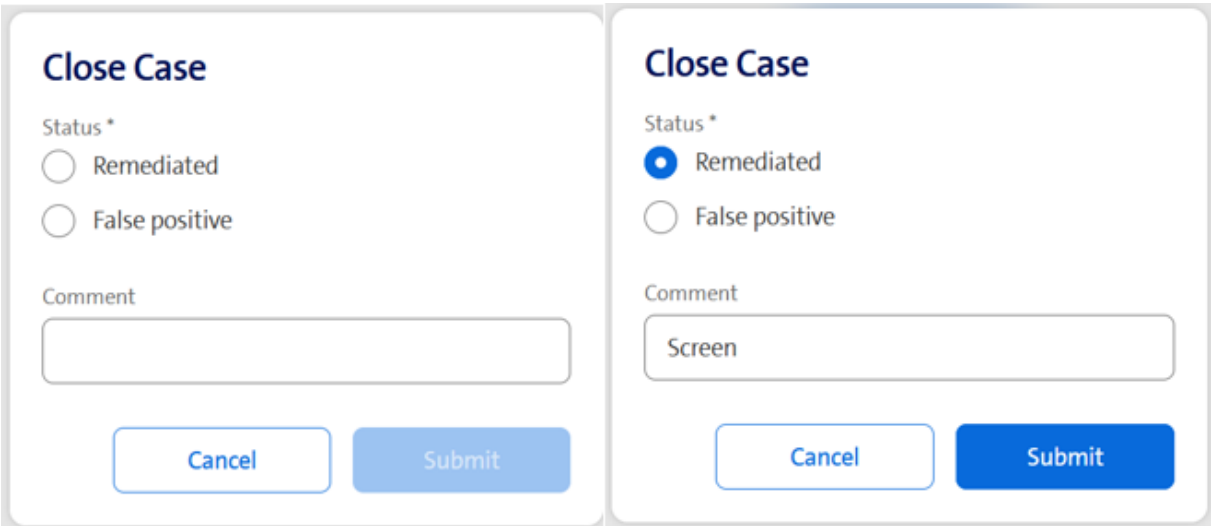


Abbildung 29 Frontend Case schliessen Modal

15.6 Softwareverwaltung

Alle Änderungen und Erweiterungen am Code wurden regelmässig und nachvollziehbar in Gitlab erfasst. Dabei habe ich auf eine klare Struktur geachtet und verständliche Commit Messages verwendet. Die Commit Nachrichten, wie auch das Branch-Naming entsprechen den teaminternen Standards, wie sie in Kapitel 3.2 beschrieben wurden.

Folgende Anforderungen abzudecken: **NF-02**

Branch naming:

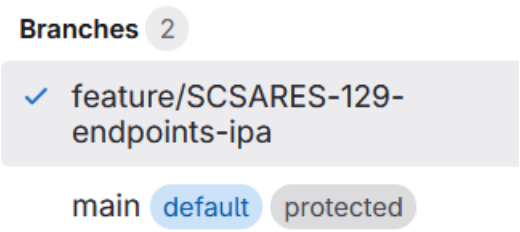


Abbildung 30 Branch naming

Commit Messages:

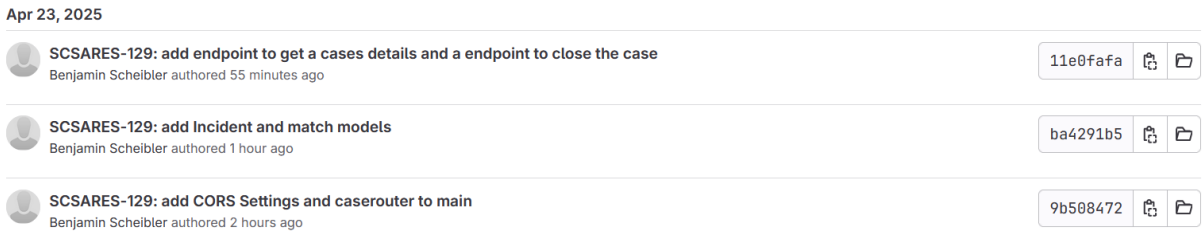


Abbildung 31 Commit Messages

Man kann auf die JIRA-Nummer klicken und gelangt direkt zum JIRA-Issue:

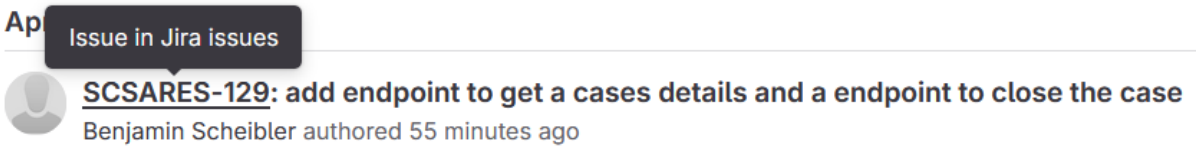


Abbildung 32 Jira-Nummer Link

16 Kontrollieren

16.1 Datenmigration

16.1.1 Funktionstest

Testdurchführung T-07 13.2.13:

1. Das Python-Skript wurde gestartet und mit dem zuvor generierten JSON-File ausgeführt.
2. Daten wurden in korrekte Datenbankstruktur geladen:

id	filename	path	status	comment	created_at
1	Lorem1.pdf	C:/Users/taascbej/SDS_temp_scan	open	[NULL]	2025-04-28 08:47:45.778
3	Lorem1.docx	C:/Users/taascbej/SDS_temp_scan	open	[NULL]	2025-04-28 08:47:45.778
4	BFH-Rechnung 900432584 vom 20.02.2025.PDF	C:/Users/taascbej/SDS_temp_scan	open	[NULL]	2025-04-28 08:47:45.778

Abbildung 33 Funktionstest Datenüberprüfung

3. Vergleich Anzahl Daten:
   Im Migrationsskript habe ich einen Counter für alle Werte eingebaut, der am Schluss die Anzahl Cases, Incidents und Matches herausgibt:

```
print(
    f"match Count: {matchCount}\nincident Count:
{incidentCount}\ncase Count: {caseCount}")
```

Konsolen Output:

```
match Count: 41
incident Count: 15
case Count: 3
```

Vergleich in der Datenbank:

select count(*) from match		select count(*) from incident		select count(*) from "case"	
Grid	123 count	Grid	123 count	Grid	123 count
1	41	1	15	1	3

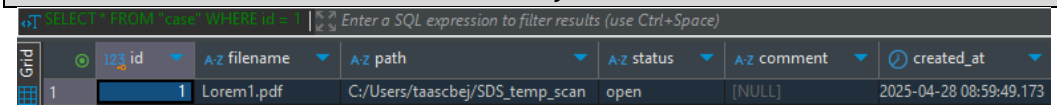
Abbildung 34 Funktionstest Datenvergleich

4. Prüfung der Datenkorrektheit:
   JSON-Report Eintrag:

```
{
      "S": "1",
      …
      "Path": "C:/Users/taascbej/SDS_temp_scan",
      "FileName": "Lorem1.pdf",
      …
      "Part": " Page 5",
      "Line": "38",
      "Position": "-10",
      …
      "Category": "PrivateNames",
      …
    },
```

SQL-Queries:

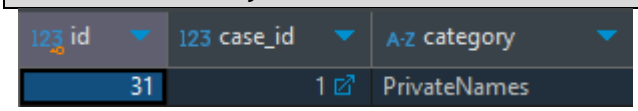
```
SELECT * FROM "case" WHERE id = 1;
```



id	filename	path	status	comment	created_at
1	Lorem1.pdf	C:/Users/taascbej/SDS_temp_scan	open	[NULL]	2025-04-28 08:59:49.173

Abbildung 35 Funktionstest Query 1

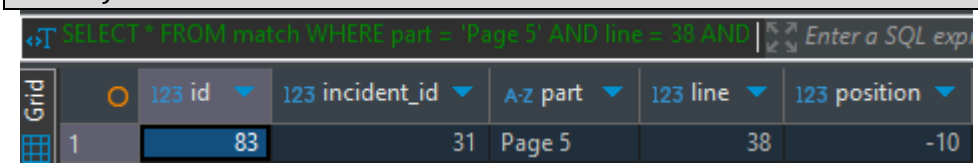
```
SELECT * FROM incident WHERE case_id = '1' AND category =
'PrivateNames';
```



id	case_id	category
31	1	PrivateNames

Abbildung 36 Funktionstest Query 2

```
SELECT * FROM match WHERE part = 'Page 5' AND line = 38 AND position
= -10;
```



id	incident_id	part	line	position
83	31	Page 5	38	-10

Abbildung 37 Funktionstest Query 3

Daten für case_id wurden korrekt übernommen. Path und Filename sind korrekt, Kategorie "PrivateNames" ist vorhanden und Match für die korrekten Part, Line und Position wurde in die Datenbank migriert.

5. Relationen prüfen

```
SELECT case_id
FROM incident
WHERE case_id NOT IN (SELECT id FROM "case")
```

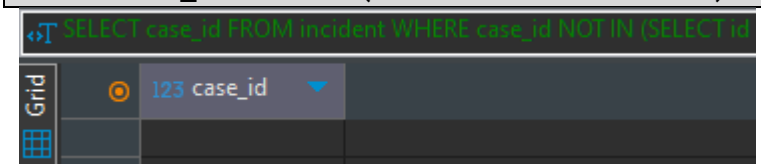


Abbildung 38 Funktionstest Relationen 1

```
SELECT incident_id
FROM match
WHERE incident_id NOT IN (SELECT id FROM incident);
```

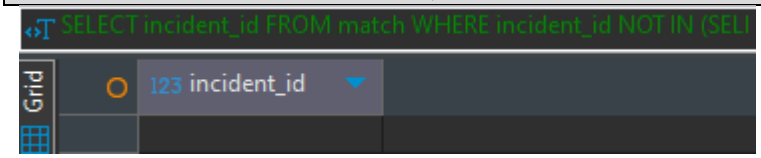


Abbildung 39 Funktionstest Relationen 2

Da keine Zeilen zurückgegeben werden, bedeutet dies, dass alle Incidents gültigen Cases zugeordnet sind und alle Matches gültigen Incidents zugeordnet sind.

Problembehebung:

Während der Tests wurden keine Unregelmässigkeiten festgestellt.

Für den Fall, dass Probleme auftreten, wurde folgendes Vorgehen definiert:

Mögliches Problem	Ursache	Vorgehen zur Behebung
Fehlende oder falsche Konvertierung von line und position	JSON-Daten als String, Datenbank erwartet Integer	Fehlerhafte Werte filtern oder Standardwert (z.B. -1) setzen und in Fehlerprotokoll aufnehmen.
Inkorrekte Pfadangaben	JSON-Fehler oder Parsing-Fehler	String-Parsing und Normalisierung im Python-Skript (z.B. .replace("\\", "/")).
Datenverlust beim Import	Fehlerhafte JSON-Formate oder Verbindungsabbrüche	Validierung des JSON vor Migration, Retry-Mechanismus für kritische Abschnitte einbauen.

Tabelle 44 Funktionstest Problembehebung

Während dem Ausführen der Datenmigration mit einem weiteren JSON-Report wurde ein Fehler im Skript entdeckt.

Es ist möglich, dass der Wert "Path" null sein kann im JSON-Report. Weil dies der Fall war, führte das zu einem Fehler bei der Migration an folgender Stelle:

```
part = entry["Part"].strip()
```

Die Funktion strip() kann nicht ausgeführt werden bei null values. Aus diesem Grund wurde das Skript folgendermassen angepasst:

```
part = entry["Part"]
    if part is not None:
        part.strip()
```

Ausserdem wurde die Datenbank so angepasst, dass null Werte in der Spalte Part erlaubt sind:

```
ALTER TABLE public."match" ALTER COLUMN "part" DROP NOT NULL;
```

16.1.2 Performancetest

Um die Performance zu testen, habe ich einen neuen SDS-Scan in einem grossen Ordner gestartet. Dieser Report gibt dann ein JSON-Report mit einer grossen Anzahl Datensätze, welche dann mithilfe des Migrationsskript in die Datenbank migriert werden.

Anzahl Datensätze:

```
Data insertion complete!
match Count: 42069
incident Count: 13779
case Count: 8549
```

Benötigte Zeit für die Migration: **1:20min**

Ressourcennutzung: CPU-Höchstnutzung lag bei **25%**, RAM-Höchstnutzung bei **3000MB**

16.2 Testdurchführung API-Endpoints

16.2.1 Alle Cases abrufen

Testdurchführung T-01 0:

```
#/testing/test_case.py

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_get_cases():
    response = client.get(
        "http://localhost:8000/api/cases?status=open")
    assert response.status_code == 200

    data = response.json()
    assert isinstance(data, list)
    for case in data:
        for field in ["id", "status", "filename", "path"]:
            assert field in case

    for case in data:
        assert case["status"] == "open", f"Case-Status ist
{case['status']}, erwartet open."

        Assert len(
            data) == 2, f"Anzahl der offenen Cases ist {len(data)},
erwartet 2."
```

Test ausführen mit `pytest app/testing/test_case.py::test_get_cases`

Resultat in der Konsole:

```
========================================= 1 passed, 5 warnings in 0.99s …
```

16.2.2 Detailansicht Cases

Testdurchführung T-03 13.2.9:

```
def test_get_case():
    response = client.get("http://localhost:8000/api/cases/2")
    assert response.status_code == 200, f"Statuscode ist
{response.status_code}, erwartet 200."

    data = response.json()
    assert isinstance(data, dict)

    for field in ["id", "status", "filename", "path", "incidents"]:
        assert field in data, f"Feld {field} fehlt in der Antwort."

    assert data["id"] == 2

    assert data["filename"] == "testfile.pdf"
    assert data["path"] == "C:/Testpath"

    incidents = data["incidents"]
    assert isinstance(incidents, list)

    for incident in incidents:
        assert "category" in incident
        assert "matches" in incident

        matches = incident["matches"]
        assert isinstance(matches, list)

        for match in matches:
            for match_field in ["part", "line", "position"]:
                assert match_field in match, f"Feld {match_field} fehlt in
einem Match."
```

Test ausführen mit `pytest app/testing/test_case.py::test_get_case`

Resultat in der Konsole:

```
========================================= 1 passed, 5 warnings in 1.20s …
```

16.3 Testdurchführung

Alle Tests wurden gemäss Beschreibung im Testkonzept durchgeführt. Folgende Tabelle zeigt die Resultate der Tests.

Test ID	Aktivität	Bestanden
T-01	Liste von Cases werden von API-Endpoint korrekt zurückgegeben	☒
T-02	Details eines einzelnen Cases werden von API-Endpoint korrekt zurückgegeben	☒
T-03	Auflistung der Cases wird korrekt dargestellt	☒
T-04	Detailansicht des Case zeigt korrekte Daten	☒
T-05	Case kann mit einem Kommentar geschlossen werden	☒
T-06	Applikation funktioniert bei grosser Datenmenge	☒
T-07	Die migrierten Daten sind vollständig und korrekt	☒

Tabelle 45 Testdurchführung

Die Durchführung von T-01 und T-02 wurde in Kapitel 16.2 genauer beschrieben.

Die Durchführung von Test T-07 funktionierte nicht von Anhieb. Probleme und Verbesserungen wurden im Kapitel 16.1.1 behandelt.

17 Auswerten

17.1 Fazit

Aufgrund von Kriterium Doc8.1 wurde das Fazit im Teil 1 der Dokumentation verfasst im Kapitel **9 Persönliches Fazit**.

17.2 Nacharbeiten

Während des gesamten Projektverlaufs – insbesondere in der Realisierungsphase – sind zusätzliche Ideen zur Erweiterung der Applikation entstanden. Auch in Gesprächen mit der VF und dem HE wurden mögliche Ergänzungen diskutiert, die ich bereits im Projektjournal festgehalten habe. Nach Abschluss der IPA sind folgende Funktionen als potenzielle Weiterentwicklungen der Applikation vorgesehen:

- Login für User
- Suchfunktion
- Pagination für Incident und Matches
- Cases nach Datum sortieren
- Geschlossene Cases nach bestimmter Zeit archivieren
- Funktion, um Case wieder zu öffnen
- Case History in der man einsehen kann, was mit dem Case passierte
    - Wie wurde der Status verändert?
    - Wer hat den Case Status angepasst?
    - Wann wurde der Case Status angepasst?
- Daten vom Scanner aus direkt in die Datenbank speichern, anstatt das Migrationsskript

18 Kriterienkatalog

Kriterium	Verweis (Kapitel)
A1	5.3, 12, 13,
A2	12.2.3, 13.4, 14.1, 19.3
A3	Gesamtes Dokument
A4	7
A5	4.4, 8, 10, 10.6, 13.6, 15.5.3 - 15.5.6
A6	4.4,7, 9, 10, 15.4.2, 16.3
A7	8, 9
A8	Gesamtes Dokument
A9	7, 13.6, 14, 14.1,
A10	4.4, 10
A11	4, 4.2, 4.3
A12-1	10.5, 10.6, 13.2.2, 13.2.5, 13.2.7 - 13.2.13
C1	12.2.1, 12.2.2
C4	13.3, 15.3, 16.1.1, 16.1.2
C5	5.2.4, 13.5, 13.4.1, 13.4.4, 13.4.5
G1	12.1, 20
G2	10.3, 12.1
G3	10.4, 10.6, 13.6
G8	13.3, 13.7, 13.7.4
G9	15.1, 15.2, 15.2.2, 15.4.2, 15.4.3, 15.5.1, 15.5.2
G10	15.415.4.2, 15.4.5-15.4.8, 15.515.6
G11	13.2, 13.2.1, 13.2.7-13.2.13, 16.2

Tabelle 46 Kriterienkatalog

19 Verzeichnisse

19.1 Tabellenverzeichnis

Tabelle 1 Informationen Lernender 14
Tabelle 2 Information Verantwortliche Fachkraft 14
Tabelle 3 Information Fachliche Unterstützung 14
Tabelle 4 Information Hauptexperte 15
Tabelle 5 Information Nebenexperte 15
Tabelle 6 Arbeitstage 23
Tabelle 7 Termine 23
Tabelle 8 Arbeitsjournal Tag 1 25
Tabelle 9 Arbeitsjournal Tag 2 26
Tabelle 10 Arbeitsjournal Tag 3 27
Tabelle 11 Arbeitsjournal Tag 4 28
Tabelle 12 Arbeitsjournal Tag 5 29
Tabelle 13 Arbeitsjournal Tag 6 30
Tabelle 14 Arbeitsjournal Tag 7 31
Tabelle 15 Arbeitsjournal Tag 8 32
Tabelle 16 Arbeitsjournal Tag 9 33
Tabelle 17 Arbeitsjournal Tag 10 34
Tabelle 18 Projektjournal erster Expertenbesuch 37
Tabelle 19 Projektjournal Besprechung Tag 1 37
Tabelle 20 Besprechung Anforderungen 38
Tabelle 21 Funktionale Anforderungen 41
Tabelle 22 Nicht-Funktionale Anforderungen 42
Tabelle 23 Legende Relevanz 42
Tabelle 24 4V case 44
Tabelle 25 4V incident 44
Tabelle 26 4V match 44
Tabelle 27 Testinfrastruktur 46
Tabelle 28 Abdeckung Anforderungen 47
Tabelle 29 Klassifizierungstabelle 62
Tabelle 30 Klassifizierung Tabellenebene 62
Tabelle 31 Klassifizierung Tabelle case 62
Tabelle 32 Klassifizierung Tabelle incident 62
Tabelle 33 Klassifizierung Tabelle match 62
Tabelle 34 Rollenbeschreibung 63
Tabelle 35 Zugriffskonzept 63
Tabelle 36 Entscheidungsmatrix Datenmigration 74
Tabelle 37 Endpoint alle Cases 75
Tabelle 38 Endpoint Case Detail 75
Tabelle 39 Entscheidungsmatrix Case schliessen 76
Tabelle 40 Endpoint Case schliessen 76

Tabelle 41 Endpoint alle Cases 85
Tabelle 42 Endpoint Case Detail 87
Tabelle 43 Endpoint Case schliessen 89
Tabelle 44 Funktionstest Problembehebung 99
Tabelle 45 Testdurchführung 104
Tabelle 46 Kriterienkatalog 106
Tabelle 47 Glossar 111

19.2 Abbildungsverzeichnis

Abbildung 1 Organigramm 17
Abbildung 2 Arbeitsplatz 18
Abbildung 3 Datensicherung Dokumentation 19
Abbildung 4 Datensicherung Zeitplan 19
Abbildung 5 Datenbank Backup 20
Abbildung 6 Projektmanagement 21
Abbildung 7 Datenbankstruktur 43
Abbildung 8 Softwarearchitektur 45
Abbildung 9 T-02 Testvoraussetzung 49
Abbildung 10 T-05 Testvoraussetzung 53
Abbildung 11 Klassifizierungsbaum 1 59
Abbildung 12 Klassifizierungsbaum 2 60
Abbildung 13 Sequenzdiagramm offene Cases anzeigen 67
Abbildung 14 Sequenzdiagramm geschlossene Cases anzeigen 67
Abbildung 15 Sequenzdiagramm Case Details anzeigen 68
Abbildung 16 Sequenzdiagramm Case schliessen 68
Abbildung 17 Komponenten 69
Abbildung 18 Schichten 69
Abbildung 19 Datenbankstruktur 71
Abbildung 20 Virtuelle Umgebung 79
Abbildung 21 Ordnerstruktur Backend 83
Abbildung 22 SwaggerUI alle Cases 86
Abbildung 23 SwaggerUI Case Detail 88
Abbildung 24 SwaggerUI Case schliessen 90
Abbildung 25 SwaggerUI Anzahl Cases 92
Abbildung 26 Frontend offene Cases 94
Abbildung 27 Frontend geschlossene Cases 94
Abbildung 28 Frontend Case Detail 95
Abbildung 29 Frontend Case schliessen Modal 95
Abbildung 30 Branch naming 96
Abbildung 31 Commit Messages 96
Abbildung 32 Jira-Nummer Link 96
Abbildung 33 Funktionstest Datenüberprüfung 97
Abbildung 34 Funktionstest Datenvergleich 97
Abbildung 35 Funktionstest Query 1 98
Abbildung 36 Funktionstest Query 2 98
Abbildung 37 Funktionstest Query 3 98
Abbildung 38 Funktionstest Relationen 1 99
Abbildung 39 Funktionstest Relationen 2 99

19.3 Quellenverzeichnis

ICT Berufsbildung Bern. Die 6-Schrittmethode IPERKA. https://www.ict-berufsbildung-bern.ch/resources/Iperka_OdA_200617.pdf

Basra, H. S. (2024, October 23). The “4Vs” - the 4 dimensions of Operations - Rowtons training. Rowtons Training. https://rowtonstraining.com/the-4vs-the-4-dimensions-of-operations/

PlantUML Web Server. (n.d.). PlantUML.com. https://www.plantuml.com/

Swisscom Wiki (2021, June 08) Klassifizieren der Vertraulichkeit von Informationen. https://wiki.swisscom.com/display/SECFX/Klassifizieren+der+Vertraulichkeit+von+Informationen

Martins, J. (2025, January 12). 7 einfache und schnelle Schritte zum Erstellen einer Entscheidungsmatrix – mit Beispielen [2025] • Asana. Asana. https://asana.com/de/resources/decision-matrix-examples

Was ist ETL? – Extract Transform Load erklärt – AWS. (2025). Amazon Web Services, Inc. https://aws.amazon.com/de/what-is/etl/

Daniel. (2023, October 30). Virtualenv Python Version: Alles über virtuelle Umgebungen. DataScientest. https://datascientest.com/de/virtualenv-python-version-alles-ueber-virtuelle-umgebungen

OpenAI (2025): ChatGPT-Kommunikation zum Thema „Textüberarbeitung“, Persönliche Kommunikation. https://chatgpt.com

OpenAI (2025): ChatGPT-Kommunikation zum Thema „PlantUML Skript“, Persönliche Kommunikation. https://chatgpt.com

OpenAI (2025): ChatGPT-Kommunikation zum Thema „Code Debugging“, Persönliche Kommunikation. https://chatgpt.com

20 Glossar

Begriff	Erklärung
B2B Kunden	Business to business Kunden, Firmenkunden
Case	Ein Case ist ein Fall. Wenn in einem File Sensitive Daten entdeckt werden, wird ein Case eröffnet.
False-Positive	Eine Fundstelle, die fälschlicherweise vom Scanner identifiziert wurde, aber gar nicht als sensitive Datenstelle angesehen wird.
Incident	Ein Incident ist ein Vorfall. Pro Case können mehrere Incidents auftreten. Ein Incident besitzt eine Kategorie, zu welchem die Matches zugeordnet sind.
Line	Beschreibt auf welcher Linie sich die Sensitive Datenstelle befindet
Match	Ein Match ist eine Fundstelle. Ein Match beinhaltet Part, Line und Position, und ist einem Incident zugehörig. So ist die Stelle im File, wo sich die Sensitiven Daten befinden, identifizierbar.
Part	Beschreibt in welchem Abschnitt des Files, sich die Sensitive Datenstelle befindet. In einem PDF ist das die Seite, in einem Excel die Arbeitsmappe.
Position	Beschreibt an welcher Stelle auf der Linie sich die Sensitive Datenstelle befindet.
Remediated	Ist Englisch und steht für behoben. In diesem Fall ist ein Case remediated, wenn es verschoben, gelöscht oder verschlüsselt wurde.
SDS	Sensitive Data Scanner – Applikation, welche Filesysteme, Datenbanken und Repositories nach Sensitiven Daten durchsucht.

Tabelle 47 Glossar

21 Anhang

21.1 Quellcode

Der Quellcode wurde als ZIP-Datei im PKORG hochgeladen. In der ZIP-Datei befindet sich ein Ordner "Frontend" und ein Ordner "Backend", welche den Quellcode beinhalten.